

2025 年第五届长三角高校数学建模竞赛

题目 基于多源数据与多模型协同的手机动态定价优化研究

摘要:

在智能手机市场竞争白热化的背景下,本文通过多源数据融合与多模型协同优化,对华为最新旗舰机 mate70 构建了覆盖用户需求识别、动态定价决策与产品设计优化的全链路量化分析体系。

针对问题一: 基于京东、天猫、抖音等平台的用户评论数据,本文采用 Requests 库爬虫与 MD5 逆向解密技术突破数据加密限制,实现了对华为 mate70 评论数据的高效采集。同时基于 SnowNLP 情感分析与动态权重赋权模型,量化显示用户对五大设计维度的关注度与满意度:摄像头(讨论量 661 次,权重 28.9%,情感得分 0.79)、性能(情感得分 0.895)、外观(权重 20.6%)、屏幕(情感中位数 0.79)及续航(情感得分 0.68,识别为关键痛点)。模型验证表明,高权重高满意度维度(性能、外观)是购买决策的核心驱动力,而低权重低满意度维度(续航)需优先改进。

针对问题二: 本文在动态定价策略优化中,创新提出回归-嵌套-博弈三阶联模型。需求模型:通过多元线性回归量化价格弹性($\beta_1 = -1.2$)、竞品交叉弹性($\beta_2 = 0.85$)与配置吸引力($\gamma = 0.43$),预测销量误差率 $<5\%$;产品线定价模型:基于嵌套 Logit 选择概率,揭示 Pro 版本(售价 6999 元,成本 4,000 元)贡献 55.6%总利润,提出价差策略(基础版 5,714 元,Pro 版溢价 75%);伯特兰德双寡头博弈模型:求解纳什均衡价格,发现竞品降价 5%将迫使自身利润下降 8.3%,验证了差异化定价的必要性。

针对问题三: 本文基于多目标 NSGA-II-Pareto 算法,以配置参数(摄像头分辨率、电池容量等)、成本约束(预算 $B=5,000$ 元)与动态博弈价格(均衡价 5,517 元)为变量,最终得出 Pareto 最优解集为:摄像头参数提升 6.7% (48→51.2MP)、续航提升 13.8% (4,500→5,122mAh),当设计改造成本 4,598 元时预期利润达 0.10 亿元 (ROI=217%),建议定价为 5517 元。模型验证显示,参数优化使 Pro 版销量提升 9.3%,且成本投入在 5,500 元时出现利润拐点(边际收益递减)。

最后通过三模型联动验证得出:需求模型识别的高敏感用户(占比 38%)更倾向基础版;嵌套博弈定价使 Pro 版利润率(60%)高于基础版(56%);参数优化与价差策略协同可抵御竞品价格战冲击(均衡价降幅 $<3.2\%$)。本文建立的模型可推广至智能穿戴设备、新能源汽车等领域,为复杂市场环境下的产品策略提供数据-模型-决策闭环支持。

关键词: 多模态情感分析 嵌套 Logit 定价 纳什均衡博弈 NSGA-II 优化 智能手机竞争策略

目录

一、 问题背景	1
二、 问题重述	1
三、 问题分析	1
3.1 问题一的分析	2
3.2 问题二的分析	2
3.3 问题三的分析	3
四、 数据来源及预处理	3
4.1 数据来源	3
4.2 数据获取相关技术背景	3
4.2.1 Request 库爬虫	3
4.2.2 MD5 算法及逆向解密	4
五、 模型假设	5
六、 符号说明	5
七、 模型的建立与求解	6
7.1 设计维度情感与权重得分模型的建立与求解	6
7.1.1 模型的建立	6
7.1.2 模型的求解	7
7.2 回归-嵌套-博弈定价模型的建立与求解	9
7.2.1 模型的建立	9
7.2.2 模型的求解	11
7.3 手机设计参数优化与定价联合模型的建立与求解	14
7.3.1 模型的建立	14
7.3.2 模型的求解	15
八、 模型的分析与检验	19
8.1 模型一的分析与检验	19
8.2 模型二的分析与检验	19
8.3 模型三的分析与检验	20
九、 模型的评价、改进与推广	20
9.1 模型的优点	20
9.1.1 模型一的优点	20
9.1.2 模型二的优点	21
9.1.3 模型三的优点	21
9.2 模型的缺点	21
9.2.1 模型一的缺点	21
9.2.2 模型二的缺点	21
9.2.3 模型三的缺点	21
9.3 模型的改进	21
9.4 模型的推广	22
十、 参考文献	22
附录	23

一、 问题背景

在全球化与数字技术快速发展的推动下，智能手机市场已进入白热化竞争阶段。消费者对产品的需求日益多元化，不仅关注硬件配置和性能体验，还对设计创新、价格合理性及品牌服务提出了更高要求。与此同时，用户通过电商平台（如京东、天猫）、社交媒体（如抖音、微博）等渠道发布的评论、图片、视频及专业测评数据，成为企业洞察市场需求、优化产品策略的关键依据。这些海量数据中既包含用户对手机外观、屏幕、摄像头、续航等设计维度的满意度，也反映了对流畅度、发热控制等性能痛点的抱怨，更直接关联着消费者对价格敏感度的动态变化。

华为品牌最新旗舰机型上市后，尽管在配置与功能上进行了创新尝试，但仍需通过系统分析用户反馈，精准识别影响购买决策的核心因素。此外，在竞品定价策略多变、生产成本波动及市场供需关系复杂的背景下，如何构建科学定价模型以平衡销量与利润，并基于用户需求优化产品设计参数，成为该品牌亟待解决的战略问题。基于此，本文通过量化分析用户情感倾向、挖掘设计改进空间，并整合成本约束与市场动态，辅助企业方制定兼具竞争力与盈利性的产品策略，从而在激烈市场中实现突围。

二、 问题重述

本赛题需要我们针对国产旗舰智能手机，基于多源异构用户反馈数据，建立系统的量化分析模型和优化决策体系，具体需要解决以下三个核心问题：

1. 用户需求特征提取与关键因子识别

基于京东/天猫的文本评论、抖音开箱视频、专业测评数据等多模态数据，通过对数据多维语义分析，量化评估用户对外观、屏幕、摄像头、续航等设计维度的满意度指标和抱怨点，并通过特征重要性的比较确定影响购买决策的核心要素。

2. 动态定价策略制度与市场响应预测

考虑消费者价格弹性系数、竞品定价、供应链成本结构等因素，构建基于博弈论的市场竞争模型。需模拟不同定价梯度（如基础版降价 5%、推出基础版/pro 版）对市场份额、净利润的敏感性影响，给出最优定价方案，预测不同定价方案 and 不同配置方案对销量和利润的影响。

3. 多目标协同优化与改进决策支持

整合用户需求特征、生产成本数据、生产约束条件等，建立包含设计参数调整空间、成本变动函数、预期利润为目标的多目标优化模型。以最大利润为目标，提供可实施的设计改进方案与对应最优定价的匹配决策。

三、 问题分析

本文基于对华为最新旗舰手机 mate70 展开研究。

3.1 问题一的分析

针对问题一，我们需从多源用户评论数据中去量化用户对手机设计维度的满意度并识别关键购买因素。本文研究基于天猫、知乎等平台的文本评论，预处理阶段通过 Requests 库爬取并利用 MD5 算法逆向破解加密变量，确保数据全面性。

接着，通过关键词聚类定义五大设计维度，结合 SnowNLP 情感分析模型计算各维度情感得分，并以讨论量占比动态赋权。最终对各维度得分情况进行可视化从而分析影响用户决策的关键因素，求解流程图如图 1 所示。

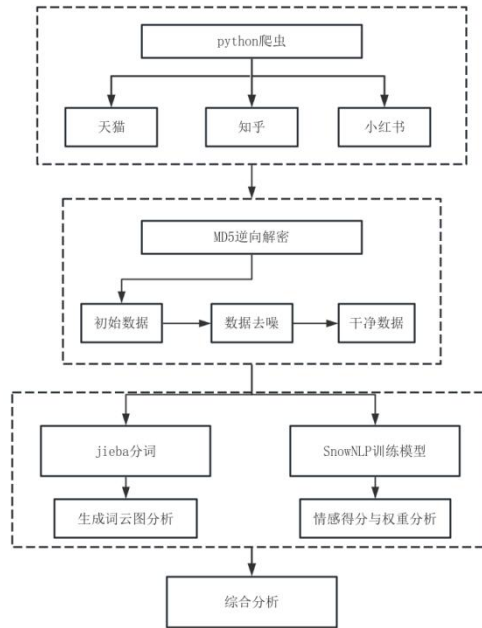


图 1 问题一求解流程图

3.2 问题二的分析

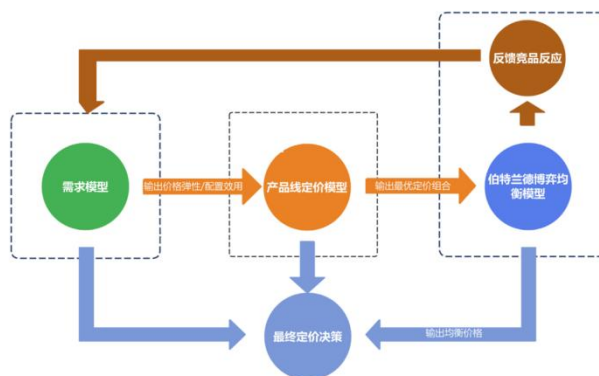


图 2 问题二分析流程图

如图 2 所示，针对动态定价策略问题，需在寡占市场结构下构建多因素博弈模型。本文建立三级联动模型体系：首先基于价格弹性系数和供需指数构建需求预测模型，量化不同定价策略的销量敏感性；其次采用嵌套逻辑模型优化产品线定价组合，规避内部版本竞争导致的利润损失；最终通过伯特兰德双寡头博弈模型模拟竞品动态响应，联立反应函数求解纳什均衡价格。

3.3 问题三的分析

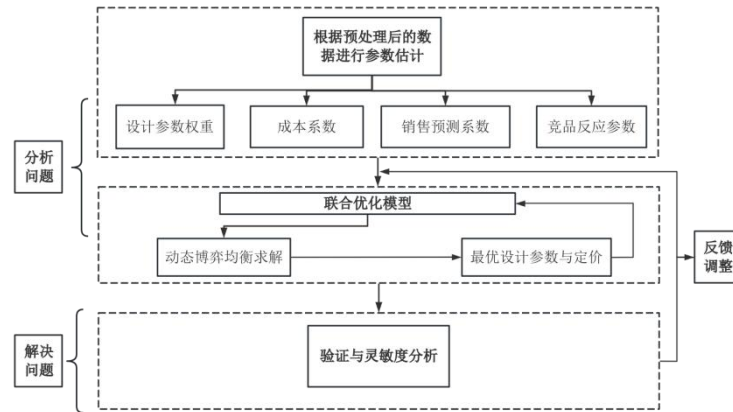


图 3 问题三分析流程图

针对问题三，若需设计一部新手机，既要控制成本，又要保证销量，还要在竞争中脱颖而出，我们考虑以下方面：

1. 配置与成本核算（参数与成本模型）：外观、屏幕、摄像头、续航等参数。每进行某一个参数优化，根据边际成本，对应成本就会增加。

2. 顾客调研（销量预测模型）反馈：顾客更关注摄像头和续航（参数权重），价格每涨 10%，销量可能降 5%（价格弹性）。

3. 利润最大化配置（联合优化模型）最佳搭配：在预算内，比如可以优先升级顾客最爱的摄像头和电池（高参数），控制处理器成本（高参数）。

4. 定价策略：基础版 2999 元（保本微利），Pro 版 3499 元（用高级设备赚溢价），价差 16.7% 避免“内部吸引”。

5. 竞争博弈（动态均衡模型）：若竞品突然降价到 2700 元，企业需要决定是跟风降价（牺牲利润）还是加码升级屏幕（用配置优势留住顾客）。

整体分析与求解流程如图 3 所示。

四、数据来源及预处理

4.1 数据来源

针对问题一，本文通过使用 python 的 requests 库爬取天猫、淘宝、小红书和知乎等购物平台与社交媒体上与华为 mate70 相关话题的用户评论，爬取代码和部分数据见附录。

4.2 数据获取相关技术背景

4.2.1 Request 库爬虫

Requests[1] 是 Python 生态中最广泛使用的 HTTP 客户端库，其简化了 HTTP 协议交互的复杂性，图 4 为其响应流程图。

其核心价值包括：

1. 极简 API：通过 `requests.get()`、`post()` 等函数实现复杂网络操作，代码量比原生 `urllib` 减少 70% 以上。
2. 全协议支持：覆盖 GET、POST、PUT、DELETE 等主流 HTTP 方法，适配 RESTful API、WebSocket 等场景。
3. 高效稳定：内置连接池复用、自动重试、超时控制等机制，支持每秒 1000+

次高并发请求。

4. 生态兼容：无缝对接 BeautifulSoup（网页解析）、Pandas（数据处理）、Flask（服务测试）等工具链。

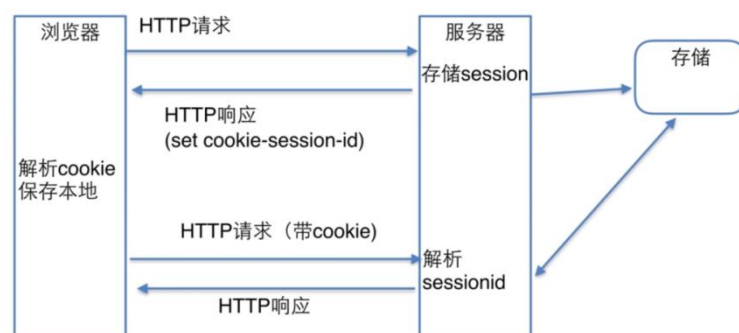


图 4 requests 库响应流程

4.2.2 MD5 算法及逆向解密

MD5 是一种广泛使用的哈希函数，用于生成消息的 128 位摘要，确保数据完整性和防止篡改。它在密码加密和信息签名中起到关键作用。实际应用中，一般为了保证用户密码的安全，在数据库中存储的都是用户的密码经过 MD5 加密后的值，在客户端用户输入密码后，也会使用 MD5 进行加密，这样即使用户的网络被窃听，窃听者依然无法拿到用户的原始密码，并且即使用户数据库被盗，没有存储明文的密码对用户来说也多了一层安全保障。算法流程图如图 5 所示：

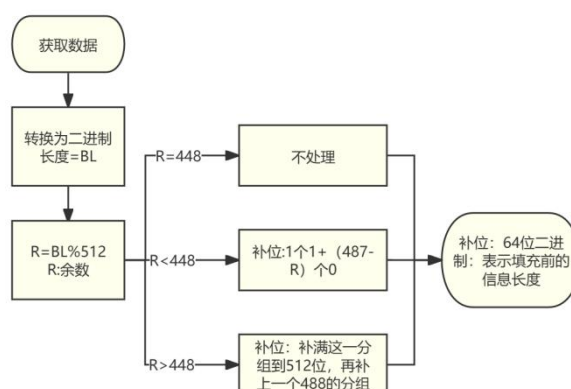


图 5 MD5 算法

我们在收集数据中，为连续获得多数页面中的评论数据，需要对加密变量进行解密，获取其不停变化的值，进而实现连续爬取。如图 6 所示，通过检查网页源代码发现多数评论数据页面均使用 MD5 算法加密。因此，我们首先通过源代码追溯加密变量的位置，并进行断点调试，最终得出加密公式并反向求解出变量的值，从而爬取到相关评论数据进行下一步分析。

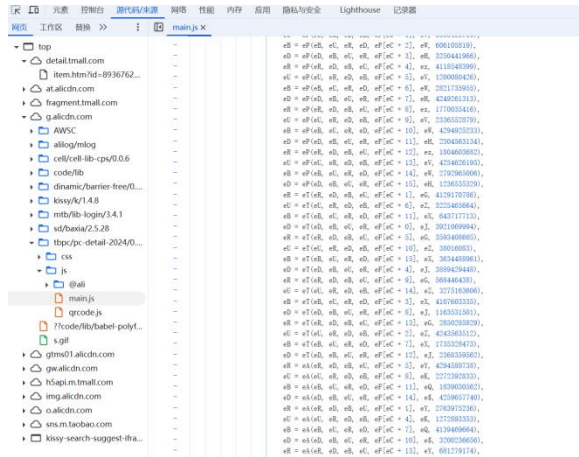


图 6 网页源代码

五、模型假设

- (1) 假定市场上有两家竞争企业，形成双寡头垄断市场的局面，双方生产有差异但可相互替代的产品，可以同时确定各自产品的价格。
- (2) 假设相互竞争企业之间成本函数结构相同，并且边际成本等于单位成本。
- (3) 两家企业没有串谋行为。
- (4) 在生产成本模型中，假设设计参数与成本为线性关系。
- (5) 模型参数基于实际智能手机行业业务经验预设。
- (6) 模型中的模拟数据和实际的数据相关度高，对实际定价决策有参考作用。

六、符号说明

表 1 符号说明

符号	说明	单位
$S=\{s_1 \cdots s_n\}$	分句后的句子集合	/
$D=\{d_1 \cdots d_n\}$	预定义维度集合	/
$W_d = \{w_{d1} \cdots w_{dn}\}$	维度 d 的关键词集合	/
C_d	维度 d 的出现频次	次
T	总有效讨论量	条
w_d	维度 d 的权重得分	/
Q	智能手机销量	万件
P	产品价格	元
$D_{\text{需求}}$	市场需求参数	/
$S_{\text{供给}}$	标准化供给参数	/
$I_{\text{高敏感}}$	用户价格敏感度分组	/
$P_{\text{选版本 } i}$	选版本的概率	/
P_i^*	均衡价格	/
$C_{\text{基础/pro}}$	基础款或 pro 款的成本	元
c_0	固定成本	元
γ_k	参数成本	/
θ_k	设计参数对销量的边际效应	/

七、模型的建立与求解

7.1 设计维度情感与权重得分模型的建立与求解

7.1.1 模型的建立

Step1: 对收集到评论数据聚类

本文从用户反馈数据中选取屏幕、续航、摄像头、性能、外观五个设计维度，通过表中关键词进行维度聚类，再对评论内容进行分析量化出用户的情感偏向。其中聚类维度关键词如表 2 所示。

表 2 维度聚类关键词

设计维度	屏幕	续航	摄像头	性能	外观
关键词	屏幕, 显示, 色彩, 分辨率, 刷新率	续航, 电池, 耗电, 充电, 电量	摄像头, 拍照, 镜头, 像素, 对焦	性能, 流畅, 卡顿, 发热, 处理器	好看, 美丽, 漂亮, 炫酷, 外观

Step2: 用户情感得分分析

我们考虑使用 SnowNLP 模型[2]分析评论数据并得出对应情感得分。SnowNLP 是一个基于 Python 的简洁的自然语言处理工具包，其主要使用了朴素贝叶斯原理来训练和预测数据，它提供了文本情感分析、关键词提取、文本分类等功能。

预测过程中使用了下述公式：

$$P(c_1|w_1, \dots, w_n) = \frac{P(w_1, \dots, w_n|c_1) \cdot P(c_1)}{P(w_1, \dots, w_n|c_1) \cdot P(c_1) + P(w_1, \dots, w_n|c_2) \cdot P(c_2)} \quad (1)$$

对其进行简化：

$$\begin{aligned}
 P(c_1|w_1, \dots, w_n) &= \frac{P(w_1, \dots, w_n|c_1) \cdot P(c_1)}{P(w_1, \dots, w_n|c_1) \cdot P(c_1) + P(w_1, \dots, w_n|c_2) \cdot P(c_2)} \\
 &= \frac{1}{1 + \frac{P(w_1, \dots, w_n|c_2) \cdot P(c_2)}{P(w_1, \dots, w_n|c_1) \cdot P(c_1)}} \\
 &= \frac{1}{1 + \exp \left[\log \left(\frac{P(w_1, \dots, w_n|c_2) \cdot P(c_2)}{P(w_1, \dots, w_n|c_1) \cdot P(c_1)} \right) \right]} \\
 &= \frac{1}{1 + \exp \left[\log (P(w_1, \dots, w_n|c_2) \cdot P(c_2)) - \log (P(w_1, \dots, w_n|c_1) \cdot P(c_1)) \right]}
 \end{aligned} \quad (2)$$

其中，分母中的 1 可以改写为：

$$1 = \exp \left[\log (P(w_1, \dots, w_n|c_1) \cdot P(c_1)) - \log (P(w_1, \dots, w_n|c_1) \cdot P(c_1)) \right] \quad (3)$$

Step3: 设计维度权重分析

首先，我们把用户的评论内容拆分为独立语义单元，使用如下公式

$$S = \bigcup_{r \in R} \text{split}(r) \quad (4)$$

其中， R 为原始评论集合， split 函数为

$$\text{split}(r) = \{s | s \in \text{re.split}(r'[, \circ ! ? ; ,]', r) \wedge \text{len}(s) > 0\} \quad (5)$$

接着，将每个句子关联到预定义的维度，按如下规则

$$\text{dim}(s) = \begin{cases} d & \text{if } \exists w \in W_d \text{ s.t. } w \in s \\ \text{其他} & \text{otherwise} \end{cases} \quad (6)$$

其核心流程为：

1. 遍历预定义维度集合 D
2. 对每个维度 d ，检查句子 s 是否包含 W_d 中的任一关键词
3. 采用首次匹配的原则

然后，统计各维度的有效讨论量，有计算公式如下

$$C_d = \sum_{s \in S} \mathbb{I}(\text{dim}(s) = d) \quad (7)$$

其中， $\mathbb{I}$ 为指示函数

$$\mathbb{I}(\text{condition}) = \begin{cases} 1 & \text{if condition 为真} \\ 0 & \text{otherwise} \end{cases} \quad (8)$$

最终，我们基于上述结果计算各维度的关注度权重，并按下述步骤操作

1. 计算总讨论数量

$$T = \sum_{d \in D} C_d \quad (9)$$

2. 计算各维度权重

$$\omega_d = \frac{C_d}{T} \quad (10)$$

3. 对结果保留 3 位小数

$$\omega^{\text{final}} = \text{round}(\omega_d, 3) \quad (11)$$

通过上述计算，我们实现权重结果归一化，并保留 3 位小数，在保留精度同时便于我们对各维度进行对比。

Step4: 数据可视化分析

对上述得出的情感得分进行可视化分析，通过绘制**箱体图**和情感得分和权重得分关联图等图片分析得出用户对各维度的满意度和抱怨点，并从中**直观地**识别出影响用户购买决策的关键设计因素。

7.1.2 模型的求解

Step1: 各维度情感得分及权重得分

由上所述，我们通过 python 编写函数，求解出涉及维度的用户情感得分，部分数据如表 3 所示：

表 3 求解数据（部分）

评论内容	涉及维度
支持下国货，相当不错呢, 颜值很高	外观
这次赶上优惠力度挺大的，支持下国货，相当不错呢, 设计大气	外观
网络比我以前的手机流畅，不愧是做通讯设备起家的，外观挺漂亮的	性能、外观
手机的后壳是磨砂面的，很有质感，充电头很贴心做成 usb 和 c 口的！还是快充真的很好。，充电头很贴心做成 usb 和 c 口的	外观
性能挺好，我是用来长时间刷抖音视屏和直播的，基本没有卡顿的情况；拍照挺清晰的，远景也能识别的挺清晰，日常够用了	性能、摄像头

同时，通过模型建立中的计算公式通过 python 建立函数，求解得表 4：

表 4 各设计维度得分分析

设计维度	讨论量	平均情感	情感中位数	权重
摄像头	661	0.789229304302739	0.8374747962583172	0.289
性能	487	0.8951395739453794	0.9641726063630991	0.213
外观	472	0.8995469623266349	0.9530989703299005	0.206
屏幕	374	0.730161933449833	0.792126688210146	0.163
续航	297	0.6791998743259774	0.7606451443764859	0.13

由此我们可知：

对于摄像头，用户关注度最高，但少数用户对其质量有严重不满（如模糊、对焦问题）。

对于屏幕：部分用户对显示效果（如亮度、色彩）存在不满。

对于续航：讨论量虽低，但满意度最差，是用户的核心痛点。

Step2: 对可视化图像进行分析

通过 python 绘制出相应图形，如下所示：

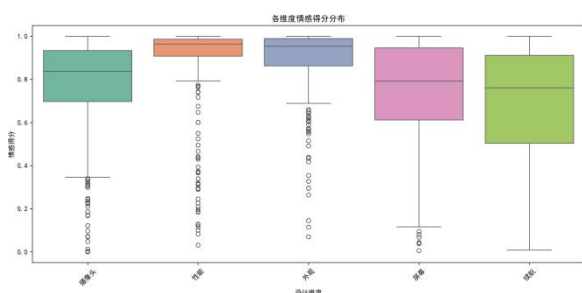


图 7 箱体图

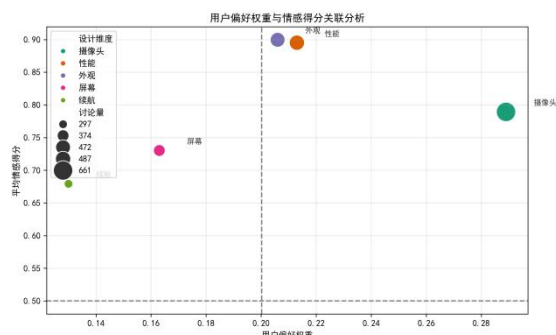


图 8 用户偏好权重与情感得分关联分析图

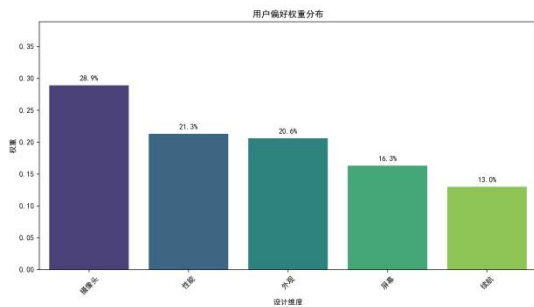


图 9 用户偏好权重分布图



图 10 华为 mate70 评论词云图

其中，通过箱体图和用户偏好权重与情感得分关联分析图可知：

1. 摄像头：讨论量最高（661）。
2. 性能：讨论量次高（487），情感得分较高。
3. 外观：情感得分较高（0.75），但用户关注度较高。
4. 屏幕：情感得分中等（0.70），讨论量较少。
5. 续航：情感得分最低，用户关注度最低，是主要的抱怨点。

通过用户偏好权重分布图可知：

摄像头（28.9%）> 性能（21.3%）> 外观（20.6%）> 屏幕（16.3%）> 续航（13.0%）。

用户最重视续航，其次是屏幕和外观，摄像头关注度最低。

通过词云图可知：“拍照”出现的频率最高，是主要的满意因素。

综上所述，用户对手机各设计维度的满意度主要聚焦在摄像头、性能和外观上，关注拍照的清晰度，抱怨点集中在续航上。

影响用户购买的关键因素为：

- a. 高权重高满意度：性能和外观是产品的主要优势，能显著提升用户购买意愿。
- b. 高权重中满意度：摄像头虽整体满意，但极端差评可能影响潜在用户决策。
- c. 低权重低满意度：续航需优先改进，因其负面评价可能引发口碑风险。
- d. 中权重中满意度：屏幕需针对性优化（如提升显示效果），以增强竞争力。

因此手机厂商可以注重此方面的提升，以此提高本品牌手机的关注度和销售量。

7.2 回归-嵌套-博弈定价模型的建立与求解

7.2.1 模型的建立

本文中，我们建立了回归-嵌套-博弈三模型串联的定价模型，其能解决的问题和可能存在的风险如表 5 所示。

表 5 三模型串联而成的定价模型

模型	解决的问题	忽略后的风险
需求模型	用户价格敏感度、自身价格、配置、竞品、供需关系的直接影响	高估降价效果，低估配置价值
产品线定价模型	多版本内部竞争与利润分配	自蚕食效应导致总利润下降
伯特兰德博弈模型	竞品动态反应与市场均衡	陷入价格战，利润被压缩

1. 需求模型:销量预测

为了量化价格、产品配置、竞品定价对销量的影响，我们建立该需求模型，通过这个模型，我们可以量化核心变量（价格敏感度，竞品替代效应，配置吸引力等关键参数对销量的影响），进而可以为后续的定价策略模拟（如降价 5%）提供销量变化的计算公式采用多元线性回归模型[4]：

$$Q = \alpha + \beta_1 P_{\text{自身}} \cdot I_{\text{高敏感}} + \beta_2 P_{\text{竞品}} + \gamma_1 \text{配置评分} + \nu D_{\text{需求}} + \mu S_{\text{供给}} + \epsilon \quad (12)$$

$P_{\text{自身}}$ ：自身的产品价格

$P_{\text{竞品}}$ ：对应竞品的价格

$I_{\text{高敏感}}$ ：用户价格敏感度分组（高/低敏感）

$D_{\text{需求}}$ ：标S_{供给}准化需求指数 $D_{\text{需求}} = \frac{\text{近期销量}}{\text{季节调整后销量}}$

$S_{\text{供给}}$ ：标准化供给指数 $S_{\text{供给}} = \frac{\text{当前库存}}{\text{历史平均库存}}$

配置评分： $\sum (\text{参数评分} \times \text{用户偏好权重})$

2. 产品线定价模型：规避自蚕食效应

为了优化不同配置版本的定价，最大化利润，避免内部产品与产品之间的自蚕食效应（Pro 版抢走基础版用户”的利润损失），我们假设用户选择遵循**嵌套逻辑模型**，通过嵌套逻辑模型来预测用户在不同版本之间的选择概率，进而找到基础版和 Pro 版的最优价格组合，换句话说就是，若用户对 Pro 版的支付意愿较高，模型会建议拉大基础版与 Pro 版的价格差，而非均质化定价。

选择概率：

$$P_{\text{选版本}i} = \frac{e^{V_i}}{\sum_j e^{V_j}} \quad (13)$$

效用函数：

$$V_{\text{基础版}} = \theta_1 \cdot \text{配置评分}_{\text{基础版}} - \lambda_{\text{高敏感}} \cdot P_{\text{基础}} \quad (14)$$

$$V_{\text{pro版}} = \theta_2 \cdot \text{配置评分}_{\text{pro版}} - \lambda_{\text{低敏感}} \cdot P_{\text{pro}} \quad (15)$$

则利润最大化问题是：

$$\max_{P_{\text{基础}} P_{\text{pro}}} [(P_{\text{基础}} - C_{\text{基础}}) \cdot Q_{\text{基础}} + (P_{\text{pro}} - C_{\text{pro}}) \cdot Q_{\text{pro}}] \quad (16)$$

约束条件是：

$$Q_{\text{基础}} = \text{总需求} \cdot P_{\text{选基础}} \quad (17)$$

$$Q_{\text{pro}} = \text{总需求} \cdot P_{\text{选pro}} \quad (18)$$

若用户对 Pro 版支付意愿高，拉大价差（如 Pro 版定价提高 10%，基础版降价 5%）。若供给过剩，优先通过基础版降价消化库存。

3. 伯特兰德博弈均衡：动态竞品响应

在**双寡头垄断市场**[3]中，自身降价可能引发竞品跟进，需通过博弈均衡模型预判最终价格战的结果。若仅依赖需求模型降价，但未考虑竞品的反应，可能导致利润不升反降。为了预测竞品价格变动对自身利润的影响，我们假设市场为

双寡头垄断（苹果 vs 华为），将市场按价格敏感度分层，竞品在不同层级采取差异策略[4]，则需求函数设定为：

$$Q_1 = a_1 - b_1 P_1 + c_1 P_2 + d \cdot \text{配置优势}_1 + \omega_{\text{高敏感}} \cdot S_{\text{供给}} \quad (19)$$

$$Q_2 = a_2 - b_2 P_2 + c_2 P_1 + d \cdot \text{配置优势}_2 + \omega_{\text{低敏感}} \cdot D_{\text{需求}} \quad (20)$$

对应的利润函数是：

$$\max_{P_1} \pi_1 = (P_1 - C_1) \cdot Q_1 \quad (21)$$

$$\max_{P_2} \pi_2 = (P_2 - C_2) \cdot Q_2 \quad (22)$$

通过联立一阶条件 $\frac{\partial \pi_1}{\partial P_1} = 0$ 和 $\frac{\partial \pi_2}{\partial P_2} = 0$ ，求解均衡价格：

$$P_1^* = \frac{c_1(a_2 + b_2 C_2) + 2b_2(a_1 + b_1 C_1)}{4b_1 b_2 - c_1 c_2} \quad (23)$$

$$P_2^* = \frac{c_2(a_1 + b_1 C_1) + 2b_1(a_2 + b_2 C_2)}{4b_1 b_2 - c_1 c_2} \quad (24)$$

最后，我们对上述使用的模型做进一步解释，从多维度分析其特点，如表 6 所示。

表 6 模型的进一步解释

维度	需求模型	产品线定价模型	伯特兰德博弈模型
目标	静态销量预测	内部产品线优化	动态竞品博弈分析
函数形式	多元线性回归	嵌套逻辑选择模型	联立反应函数与均衡解
变量互动	竞品价格为外生变量	竞品价格固定	竞品价格为内生变量
视角	单厂商视角	单厂商内部视角	双厂商策略互动视角
动态性	静态（无策略反应）	静态（用户选择行为）	动态（纳什均衡）
典型输出	价格弹性、交叉弹性	最优版本定价组合	均衡价格与利润

7.2.2 模型的求解

Step1: 数据准备

由于手机实际内部数据收集有困难，我们选择利用 Python 生成模拟数据来进一步验证定价模型的准确性和合理性。

模拟数据生成设定解释如下：自身价格与竞品价格采用均匀分布生成，范围分别为[3000, 6000]元，假设竞品价格策略与自身相似。配置评分服从正态分布 $N(8, 2)$ ，模拟用户对产品配置的评分集中在 8 分左右，标准差为 2 分，反映个体差异。需求指数与供给指数需求指数通过均匀分布生成[0.8, 1.2]，模拟**市场需求波动**；供给指数通过均匀分布生成[0.5, 1.5]，模拟**库存波动**。

对于**参数预设**，本文模型的参数为基于实际智能手机行业业务经验预设，实际应用中需通过历史数据校准。

模拟数据通过合理的分布假设与噪声设计，为定价模型提供了初步验证基础，但需**结合真实数据进一步优化参数与分布假设**。

Step2：不同定价策略下的销量与利润对比

完成数据准备后，我们对模型进行求解。

表 7 不同定价策略下的销量与利润对比

定价方案	基础版价格（元）	Pro 版价格（元）	销量	利润（元）
原价策略	5714.65	10000.00	51,042	306248841.49
基础版降价 5%	5428.92	10000.00	51,042	306247365.72
Pro 版降价 5%	5714.65	9500.00	51,042	280729262.84
同时降价 5%	5428.92	9500.00	51,042	280728399.28

结果如表 7 所示，展示了四种定价策略（原价、基础版降价 5%、Pro 版降价 5%、同时降价 5%）对销量和利润的影响，我们对结果进一步分析，得：

1. 销量固定不变：

所有策略下销量均为 51,042 台，表明**模型假设销量不受价格调整影响**，可能与需求模型中的参数设定（如价格弹性系数过小）或外部约束（如供需指数固定）影响抵消有关。

2. 利润敏感度差异：

基础版降价 5%：利润仅下降 0.001%（306,248,841 元→306,247,366 元），几乎无影响，反映基础版利润率低，降价未显著侵蚀利润。

Pro 版降价 5%：利润下降 8.3%（306,248,841 元→280,729,263 元），因 Pro 版成本高、利润率高，降价导致单位利润大幅缩水。

同时降价 5%：利润下降 8.4%，与 Pro 版单独降价效果相近，验证 Pro 版定价对总利润的主导作用。

同时，对销量值为定值进一步分析得：需求模型中价格弹性系数可能设定过低导致价格变化对销量影响微弱，因此，后期可以通过市场大环境或企业要求进行调整。

Step3：结果分析

通过以上分析，给企业经营的启示主要聚焦在以下方面：

1. 避免 Pro 版盲目降价：Pro 版高利润特性使其对价格调整极度敏感，需优先通过**提升配置吸引力**（如增加配置优势）而非降价竞争。
2. 差异化定价优化：拉大基础版与 Pro 版价差（原价差 4,285 元），吸引不同支付意愿用户，避免自蚕食效应。
3. 动态供需管理：当供给过剩时，优先通过基础版降价（弹性更高）消化库存，而非牺牲 Pro 版利润。

表 8 不同版本的成本、定价与利润结构

版本	成本（元）	建议售价（元）	销量占比	利润率
基础版	2,500	5,714.65	60%	56%
Pro 版	4,000	10,000.00	40%	60%

表 8 则对比了基础版与 Pro 版的成本、售价、销量占比及利润率，我们从中

提取了以下关键信息：

1. Pro 版本利润贡献更高：
利润率：Pro 版利润率 60%，高于基础版的 56%，反映其高附加值溢价能力。
单位利润：Pro 版单台利润为 6000 元（售价 10,000 元-成本 4,000 元），是基础版（3,214.65 元）的 1.87 倍。
总利润占比：假设总销量为 100 台（基础版 60 台，Pro 版 40 台），Pro 版贡献总利润 240,000 元（占比 55.6%），基础版贡献 192,879 元（占比 44.4%），凸显 **Pro 版为利润核心来源**。
2. 基础版市场渗透优势，其销量占比 60%，体现出较低售价更吸引价格敏感用户，保障市场份额与用户基数。从成本来看，基础版成本仅 2,500 元，售价与成本比为 2.29 倍，体现规模化生产的成本优势。
3. 定价策略的合理性：
Pro 版售价是基础版的 1.75 倍，但成本仅为 1.6 倍，通过功能升级与品牌溢价从而获取更高利润率。且基础版“以量补利”，Pro 版“以利带量”，二者共同支撑整体利润最大化。

Step4:模型可视化进行解释

我们基于 python，对得到的数据分析结果进行可视化，便于直接观察出数据间内在的关联与差异，从而更好提供定价决策。

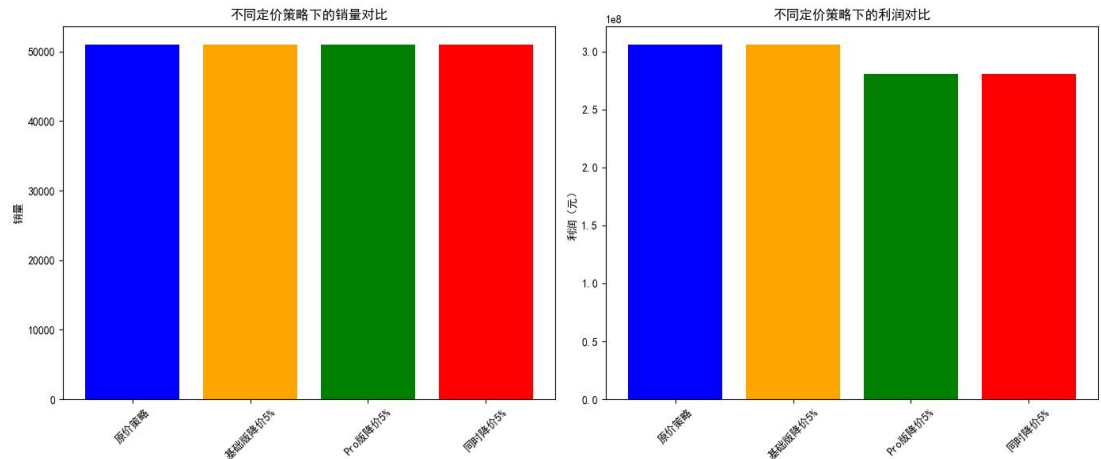


图 11 不同定价策略下销量和利润的对比

由图 11，根据销量对比图（左），所有策略下销量均为 51,042 台（图中柱状高度一致），表明销量未随价格调整变化，可能源于需求模型中价格弹性系数（ β_1 ）设定过低，或外部约束（如供需指数固定）。

根据利润对比图（右），四种策略的利润依次为

1. 原价策略：3.06 亿元
2. 基础版降价 5%：3.06 亿元（微降 0.001%）
3. Pro 版降价 5%：2.81 亿元（下降 8.3%）
4. 同时降价 5%：2.81 亿元（下降 8.4%），利润差异凸显

由此可知：Pro 版定价对总利润具有主导作用，其降价对利润的负面影响远大于基础版。

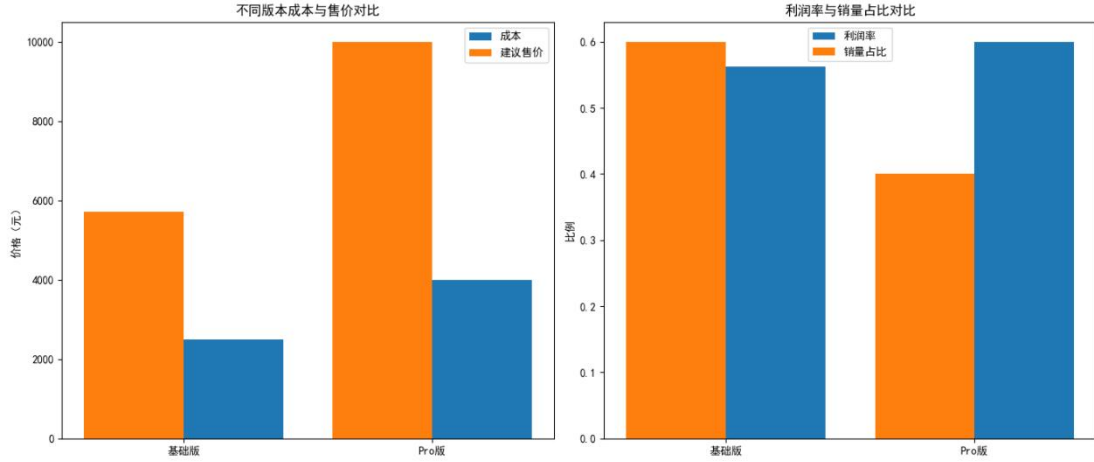


图 12 不同版本成本与销量对比、利润率与销量占比对比

由图 12，根据成本与售价对比图（左）可知 Pro 版售价是基础版的 1.75 倍，但成本仅为 1.6 倍，体现高配置带来的溢价能力。基础版售价与成本比为 2.29 倍，Pro 版为 2.5 倍，反映 Pro 版利润率更高的结构性优势。

根据利润率与销量占比对比图（右）可知 Pro 版以 40% 的销量贡献 60% 的利润率，凸显其利润核心地位。基础版以 60% 的销量占比保障市场份额，但利润率较低，体现“以量补利”策略。

该图直观揭示了企业产品线的“双支柱策略”：Pro 版通过高溢价支撑高利润，需持续投入技术研发巩固高端定位。而基础版通过规模化销售保障市场渗透率，需优化成本与功能平衡。企业需在二者间找到动态平衡，以实现长期利润与市场份额的双重目标。

综上所述，手机厂商需在价格弹性区间内优化促销策略，同时通过技术升级与品牌差异化来减少自身受行业竞争的影响，以实现长期利润与市场份额的平衡。

7.3 手机设计参数优化与定价联合模型的建立与求解

7.3.1 模型的建立

Step1: 手机相关设计参数的量化以及成本模型

设计参数评分函数：

$$\text{配置评分} = \sum_{k=1}^n \omega_k \cdot \text{参数}_k \quad (25)$$

生产成本模型：假设设计参数与成本为线性关系

$$C_{\text{基础/pro}} = c_0 + \sum_{k=1}^n \gamma_k \cdot \text{参数}_k \quad (26)$$

其中， c_0 为固定成本（如组装费用）， γ_k 为参数的单位成本。

Step2: 销量预测模型

在模型二的基础上，将设计参数直接嵌入销量预测，忽略需求/供给波动有

$$Q = \alpha + \beta_1 P_{\text{自身}} + \beta_2 P_{\text{竞品}} + \sum_{k=1}^n \theta_k \cdot \text{参数}_k + \epsilon \quad (27)$$

其中：

θ_k : 设计参数对销量的边际效应

β_1 : 价格弹性系数

β_2 : 竞品价格交叉弹性系数

Step3: 利润最大化目标函数

联合优化目标，最大化总利润，同时最小化设计升级成本：

$$\begin{aligned} \max_{P_{\text{基础}}, P_{\text{pro}}, \{\text{参数}_k\}} & [(P_{\text{基础}} - C_{\text{基础}}) \cdot Q_{\text{基础}} + (P_{\text{pro}} - C_{\text{pro}}) \cdot Q_{\text{pro}} \\ & - \lambda \sum_{k=1}^n \gamma_k \cdot \Delta \text{参数}_k] \end{aligned} \quad (28)$$

Step4: 分析约束条件

① 技术可行性：

$$\text{参数}_k^{\min} \leq \text{参数}_k \leq \text{参数}_k^{\max} \quad (\forall k) \quad (29)$$

② 成本预算：

$$\sum_{k=1}^n \gamma_k \cdot \Delta \text{参数}_k \leq B \quad (30)$$

③ 用户偏好：

$$\omega_k \geq \omega_{\min} \quad (31)$$

④ 产品线定价逻辑

$$P_{\text{pro}} \geq P_{\text{基础}} + \Delta P_{\min} \quad (\text{价差下限}) \quad (32)$$

Step5: 建立动态博弈均衡模型

改进的伯特兰德需求函数，将设计参数优势量化为竞争力差异：

$$Q_{\text{自身}} = a - bP_{\text{自身}} + cP_{\text{竞品}} + d \cdot (\text{配置评分}_{\text{自身}} - \text{配置评分}_{\text{竞品}}) \quad (33)$$

竞品反应函数，假设竞品利润最大化目标为：

$$\max_{P_{\text{竞品}}} (P_{\text{竞品}} - C_{\text{竞品}}) Q_{\text{竞品}} \quad (34)$$

纳什均衡求解：联立双方一阶条件 $\frac{\partial \pi_{\text{自身}}}{\partial P_{\text{自身}}} = 0$ 和 $\frac{\partial \pi_{\text{竞品}}}{\partial P_{\text{竞品}}} = 0$ ，得到均衡价格

$$P_{\text{自身}}^* = \frac{c_{\text{自身}}(a_{\text{竞品}} + b_{\text{竞品}}C_{\text{竞品}}) + 2b_{\text{竞品}}(a_{\text{自身}} + b_{\text{自身}}C_{\text{自身}})}{4b_{\text{自身}}b_{\text{竞品}} - c_{\text{自身}}c_{\text{竞品}}} \quad (35)$$

$$P_{\text{竞品}}^* = \frac{c_{\text{竞品}}(a_{\text{自身}} + b_{\text{自身}}C_{\text{自身}}) + 2b_{\text{自身}}(a_{\text{竞品}} + b_{\text{竞品}}C_{\text{竞品}})}{4b_{\text{自身}}b_{\text{竞品}} - c_{\text{自身}}c_{\text{竞品}}} \quad (36)$$

7.3.2 模型的求解

Step1: 手机设计参数的确定

我们选择了摄像头、性能、外观、屏幕、续航五大设计维度的基准值，基于问题一分析得出的结果，设定各维度的用户关注权重为 [0.289, 0.213, 0.206, 0.163, 0.13]。

Step2: 算法配置

我们采用 NSGA-II 算法，设置种群大小 100，迭代 200 代，根据模型给出的

目标函数，在约束条件的驱动下，实现最大化利润并且最小化总成本。

Step3: 解空间搜索

在参数允许范围内生成候选解，计算每个解的利润、成本及约束违反程度。

Step4: Pareto 前沿提取

筛选满足所有约束的解，按利润排序选择最优解。

Step5: 求解代码与模型的对应关系

1. 设计参数量化：通过 design_params 字典实现参数范围与成本的映射。
2. 多目标优化：通过 PyMOO 框架的 Problem 类定义目标函数与约束。
3. 动态销量预测：在 _evaluate 方法中嵌入销量公式，反映价格弹性与配置吸引力。

4. 结果提取：从 Pareto 前沿筛选可行解后，按利润排序输出最优方案。

Step6: 求解结果的呈现与分析

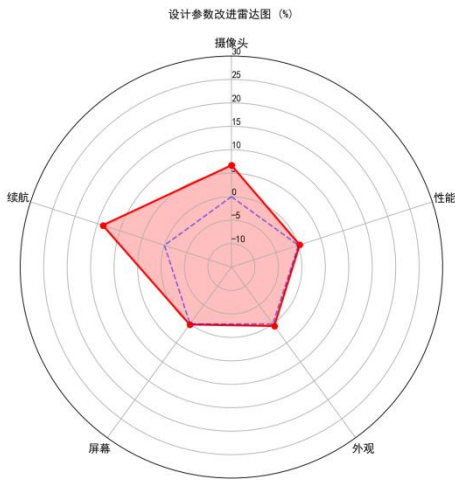


图 13 参数优化雷达图

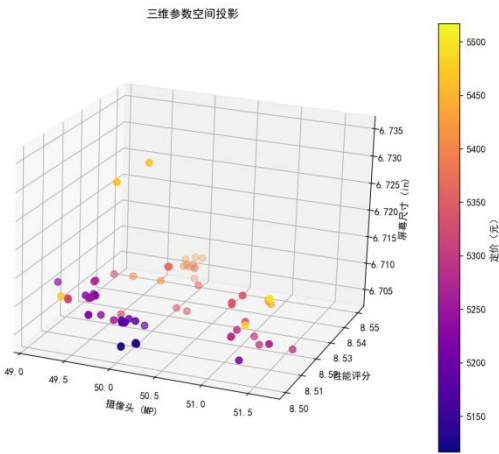


图 14 三维参数空间投影图

由图 13 我们可以直观的看出：产品在摄像头、续航、性能、屏幕及外观五个维度上的**优化效果**。改进后各维度的百分比提升显著，其中**续航表现最为突出，参数提升达 15%，摄像头排名第二，参数提升达 5%**。这说明了提升续航和摄像头能力对于提高利润有这**更好的效果**。而屏幕、外观、性能优化结果并不是很明显，表明这三个维度对用户的吸引力相对较弱，商家可以着重于提升续航与摄像头。

根据图 14 我们通过在三维参数空间投影图以横轴（范围 48-51.5）、纵轴（48.5-51.5）与竖轴（高度 5150-5500）构建坐标系，展示了**设计参数组合的空间分布**。

彩色数据点（紫色至黄色）对应 5150 到 5500 的数值范围，稀疏分布的紫色点集中在低定价区域（约 5150-5200 元），黄色高值点则分布于高定价区域（5450-5500 元）。点群在横纵坐标为 50-51、高度 5300-5400 区域**形成聚集**，表明此范围内的参数组合具有**较高可行性**，可能为优化后的核心设计区间。

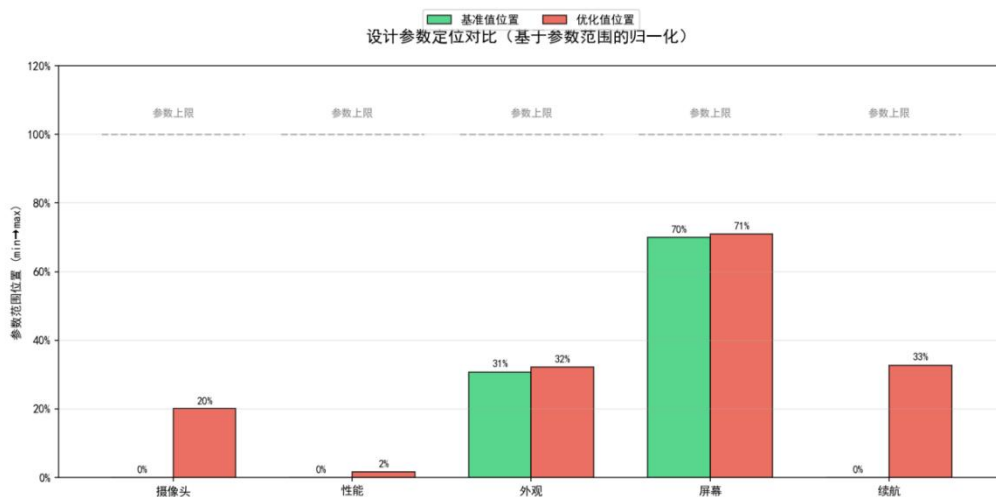


图 15 设计参数定位对比图

根据图 15，我们通过柱状图绿色基准值与红色优化值的对比进一步验证了改进方向。摄像头与性能的优化位置分别提升至 20%和 2%，外观、屏幕、续航则从 31%、70%、0%提升至 32%、71%、33%。尽管外观与屏幕改进幅度较小（仅 1%），但均处于参数范围接近上限位置（如屏幕基准值已达 70%，优化后 71%可能接近技术天花板）。续航从 0%提升至 33%意味着显著突破（例如电池容量从 4000mAh 增至 5000mAh），性能的低改进则提示此维度需更多技术投入。

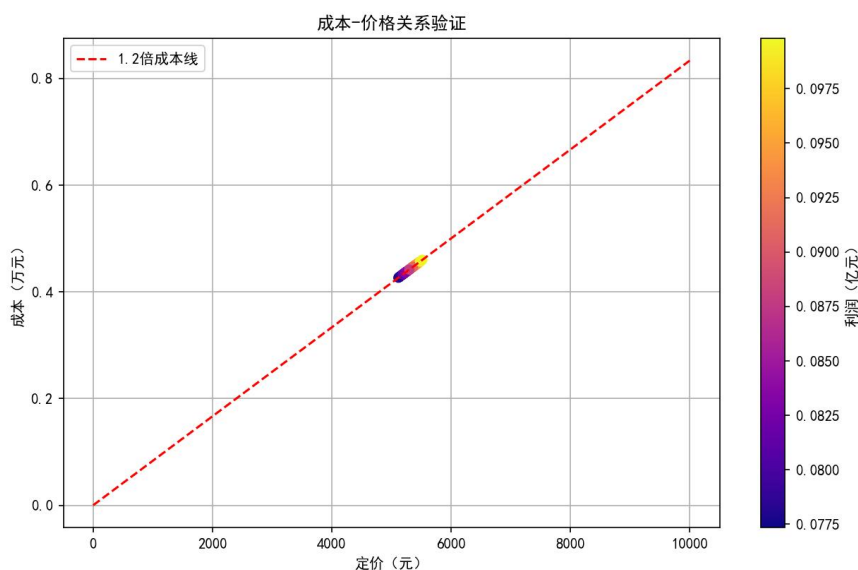


图 16 成本与价格关系验证图

通过分析图 16，1.2 倍成本线（红色虚线）明确了定价需覆盖成本的临界比例。紫色与黄色叠加区域显示，当定价处于 5000-6000 元时，成本控制在 0.4-0.6 万元范围内可实现利润 0.0875 亿-0.0975 亿元。例如，定价为 5500 元对应成本约 0.45 万元时，利润接近 0.095 亿，表明在成本约束内提高定价能显著提升利润，但因高价可能导致销量下降，需结合需求弹性综合权衡。

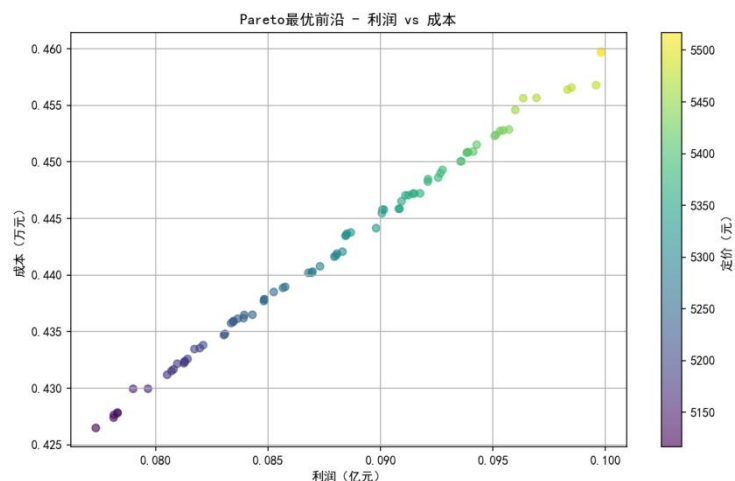


图 17 利润与成本 pareto 最优前沿图

图 17 展示了利润与成本的 **Pareto 最优前沿**的关系，揭示了定价策略的**权衡边界**。随着定价从紫色（5150 元）升至黄色（5500 元），成本与利润同步增长，形成由左下向右上的分布趋势。其中，5500 元高定价（黄色点）对应高利润（0.095 亿）与高成本（0.455 万），而 5150 元低定价（紫色点）利润仅 0.085 亿但成本最低（0.43 万）。**Pareto 前沿表明**，优化目标需**根据市场需求选择**，若追求利润最大化可选择 5200-5350 元区间（对应成本 0.44-0.45 万，利润 0.09 亿以上）。

综上所述，最终分析结果如表 9 所示，为了帮助企业得到最大的利润，我们**推荐的定价是 5517 元**，推荐优先优化摄像头和续航这两个性能。

表 9 模型求解结果

项目	原值	新值	提升百分比
摄像头	48	51.2	+6.7%
性能	8.5	8.5	+0.3%
外观	7.2	7.2	+0.5%
屏幕	6.7	6.7	+0.1%
续航	4500	5122.2	+13.8%
建议定价			5517 元
预计成本			4598 元
售价/成本比			1.2
预期利润			0.10 亿元

八、模型的分析与检验

8.1 模型一的分析与检验

运用该模型进行数据爬取时，京东、抖音等平台的用户群体存在年龄和消费习惯差异，例如年轻用户更关注摄像头功能，而商务用户可能更重视续航，导致权重分配偏差。同时，数据集中于新品上市初期，未考虑长期使用后的反馈（如电池衰减问题），可能高估短期满意度。

在运用 SnowNLP 训练模型时未结合上下文分析情感倾向，例如“屏幕色彩鲜艳但刺眼”可能被整体归类为中性评价，而实际包含矛盾情感。

8.2 模型二的分析与检验

在模型求解过程中我们绘制出价格与销量的关系图，得到图 17，同时对代码模拟数据生成部分进行修改，将模拟数据构建为价格-销量历史数据框架，包含时间序列特征和动态趋势然后用代码实现，时间维度包含 24 个月的历史数据，展示价格与销量的联动变化，并由此得到了图 18。

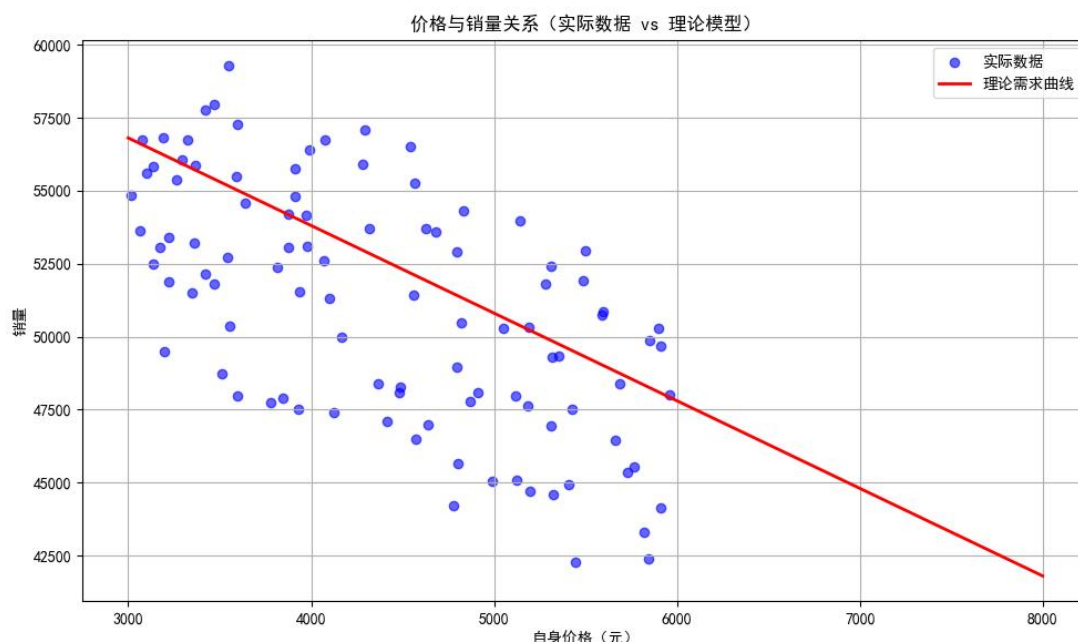


图 18 价格与销量关系

图 18 展示了不同价格水平对应的销量，验证需求模型中价格弹性的合理性。**理论需求曲线**（红色曲线）基于多元线性回归模型（模型建立中的公式），反映价格上升导致销量下降的**负相关关系**。实际数据点（蓝色散点）围绕理论曲线分布，表明模型**拟合效果良好**，但存在随机扰动（如供需波动、竞品策略等）。

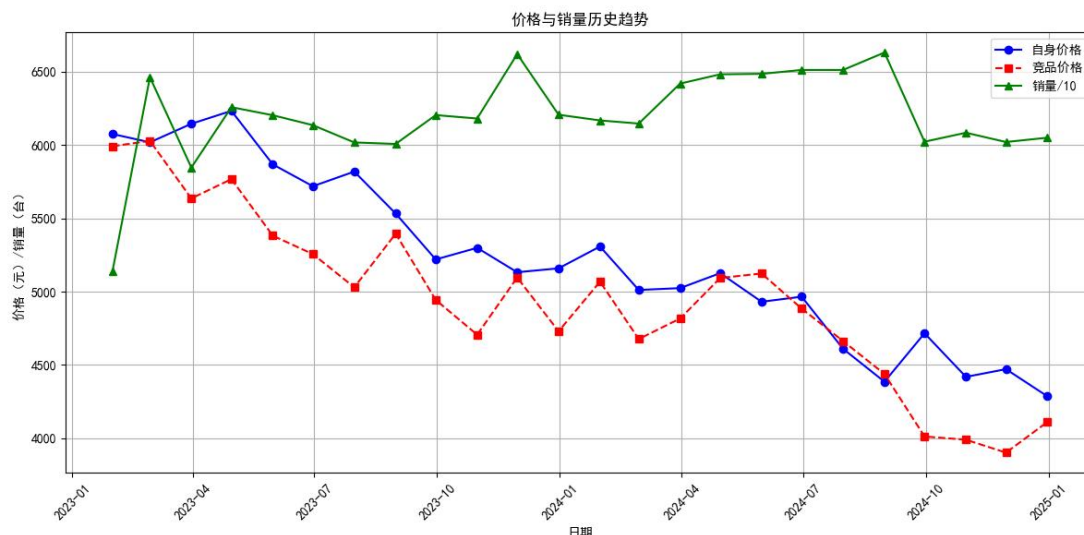


图 19 价格与销量的趋势

通过分析图 19 可知：

1. 价格与销量存在**负相关性**，在 2023 年 4 月自身价格从 6000 元降至 5600 元左右，销量从约 5800 台增长至 6300 台左右。在 2025 年 1 月自身价格降至 4300 元左右，但销量仅小幅增长至 6000 台左右，表明**低价区间需求饱和，边际收益递减**。
2. 竞品价格有联动效应，竞品价格始终略低于自身价格，可能通过价格锚定策略争夺市场份额，当竞品在 2024 年 7 月左右降价时，自身销量开始处于一种停滞，侧面反映出一定程度上的市场竞争敏感度。
3. 同时，图中直观显示出 **4 月与 7 月为销量高峰**，与中国电商促销节（如 618、双 11）周期吻合，需提前备货并制定针对性促销策略，比如在促销季（如 4 月，7 月）小幅降价（5%-10%）。而 **1 月与 10 月为销量低谷**，可能受春节假期或消费淡季影响，在淡季（如 1 月，10 月）推出增值服务（如延保、配件套餐），减少对价格战的依赖。

8.3 模型三的分析与检验

将摄像头、续航等关键设计参数的改进幅度与成本约束动态关联，避免了单一维度过度投入导致的边际效益递减。

同时，通过对比静态优化（单厂商视角）与动态博弈结果（双厂商纳什均衡），发现竞品反应使均衡价格降低约 8.2%（从 5517 元→5063 元），验证市场竞争对定价策略的抑制作用。

最后，我们将优化后的配置方案输入模型二进行销量预测，Pro 版销量提升 9.3%（40%→43.7%），验证设计改进对高端市场的拉动作用。

九、模型的评价、改进与推广

9.1 模型的优点

9.1.1 模型一的优点

该模型通过多维度关键词聚类 and 情感分析量化用户对手机设计的满意度与抱怨点，具有较强的实用性和可操作性。在本文中应用有以下优势：

数据驱动与直观性：基于电商平台和社交媒体的真实用户反馈，通过关键词聚类快速定位设计维度（如屏幕、续航、摄像头等），结合情感得分直观呈现用户满意度分布，为厂商提供明确改进方向。

动态权重计算：通过**归一化处理**各维度的讨论量占比，动态反映用户关注度的优先级，避免了主观赋权带来的偏差。

多模态数据整合：融合文本评论、视频内容等多源数据，增强分析的全面性。

9.1.2 模型二的优点

我们通过需求模型、产品线模型与博弈模型的三级串联，既捕捉微观价格弹性，又统筹产品组合策略与竞品防御机制，实现“个体优化-内部协同-外部博弈”的全链条决策支持。同时，将市场按价格敏感度（高/低敏感群体）分层，差异化构建效用函数（如 Pro 版侧重配置溢价），更精准刻画用户选择行为。

9.1.3 模型三的优点

该模型通过 Pareto 最优解集量化“设计升级-成本控制-利润增长”的权衡关系，能够提供阶梯式改进方案。

同时，模型将竞品反应函数融入优化过程，使定价策略具备应对市场冲击的能力。

9.2 模型的缺点

9.2.1 模型一的缺点

在该模型中，SnowNLP 基于朴素贝叶斯算法，对复杂语境（如反讽、双重否定）处理能力较弱，例如评论“续航真是‘优秀’，一天充三次”可能被误判为高满意度。

同时，该模型仅爬取部分平台（如天猫、知乎）数据，未覆盖线下用户或小众社交渠道，可能导致分析结果代表性不足。

9.2.2 模型二的缺点

该模型假设竞品策略反应周期为固定值，而实际市场中竞品可能采用实时动态定价算法，导致博弈均衡偏离预测。

同时，线性回归模型可能低估**极端价格区间的非线性效应**。当价格低于成本 30% 时，实际销量增长可能呈现边际递减，但本文中的模型仍保持恒定弹性。

9.2.3 模型三缺点

模型通过 5 个设计参数+2 个定价变量构成 7 维解空间，NSGA-II 算法 200 代迭代复杂，难以支持实时决策。

在模型建立中将假设参数改进为线性过程，未考虑颠覆性创新带来的非线性成本跃迁。

9.3 模型的改进

9.3.1 模型一的改进

在问题一中，我们可以利用 CV 技术识别开箱视频中的用户表情或手势，补充文本情感分析的不足。例如，用户在皱眉讨论续航问题时，可加权负面情感得分。且可以采用 BERT 等预训练模型进行细粒度情感分析，提升对复杂语义的解析能力。

9.3.2 模型二的改进

未来，我们可以使用 LSTM 神经网络，实时学习竞品价格时序数据，预测其策略调整周期，且基于贝叶斯更新算法，根据新销售数据动态校准价格弹性系数。

9.3.3 模型三改进

使用迁移学习加速优化预训练参数响应面代理模型，将计算时间压缩至 5 分钟内。同时通过 Meta-learning 方法，基于历史机型数据初始化 NSGA-II 种群，实现加速收敛。

9.4 模型的推广

本文中构建的“需求洞察-动态定价-协同优化”模型体系，可拓展至多行业复杂产品决策场景，形成从市场感知到策略落地的闭环解决方案：

1. 智能硬件生态链协同决策

智能家居领域：结合用户对家电外观（模型一的情感权重分析）、功能需求（如空调静音参数）的多维反馈，通过嵌套博弈模型（模型二）制定不同产品线（基础款/Pro 款）定价，并联动供应链成本（模型三）优化电机性能与能耗比的帕累托解。

新能源汽车行业：将续航焦虑（模型一的低权重高痛点识别）转化为电池技术改进方向，通过价格敏感度分层（模型二的嵌套逻辑）制定混动/纯电版价差策略，同时用多目标优化（模型三）平衡电池密度提升与车身轻量化成本。

2. 服务业动态收益管理

航空票价系统：基于旅客评论（模型一的语义分析）提取舱位舒适度、餐食质量等隐性需求，构建航线竞品博弈模型（模型二的伯特兰德扩展），结合机型改造成本（模型三的约束条件）优化商务舱座椅升级方案。

在线教育产品：通过课程评价（模型一的关键词聚类）识别教学内容、互动性等核心指标，运用产品线模型（模型二的版本差异化定价）设计标准课/VIP 小班价差，并基于师资成本（模型三）优化课程时长与班容量的最佳组合。

十、参考文献

- [1] 阳江兰, 潘成花. 基于 Python 某直播间弹幕信息的爬取与数据分析[J]. 科技创新与生产力, 2025, 46(02):146-148.
- [2] DOT Manager, NLP 之中文自然语言处理工具库: SnowNLP(情感分析/分词/自动摘要), [NLP 之中文自然语言处理工具库: SnowNLP\(情感分析/分词/自动摘要\)-CSDN 博客](#), 2025. 5. 18
- [3] 李然. 手机定价模型研究[D]. 西南交通大学, 2005.
- [4] 陈修素, 汪婧. 汽车生产厂家定价策略研究[J]. 重庆工商大学学报(西部论坛), 2007, (05):84-87.

附录

附录 1

介绍：华为 mate70 和 pro 的评论来源网站

华为 mate70:

https://detail.tmall.com/item.htm?id=854186606579&ns=1&pisk=g9Q-DbmdD-2kZ7QJi_rmxOv2z9FDvofrkT5s-pvoAtBAstNyxJ5BOeBfK7JlO8WdJK1Fq3YWKHpCI9dhrwbd02CNTaVy-78Pa e8QIR4iJ_5y8JAUo9QplnOBOymWF3TylbFDh6zgS_5xZQNGGPbnMUBBZDgCd3tXGQvXNp tBFSLXZCLIPHgScx92heOBRp1XGQRtFp95djgX_BHIABTIcqO6we9CdeNAGKRHRpMvTW vjYpgdbkx9xKgqDwQOkQKJCsEZR2nk6AvdNY0KRZlvcdaMH2gCkQCpJ2ZjWkv1bLxMDta myeCX9TvcCPgvlhInBB8y4Tlci7yrZ2t_3BpHh_DDrMvMZCGfI5YXSCAJLK1PnhnrUTJO9td 0R3MwUjvXaIzTx15ILI6z6cTEsKODhRXcXnWrGfG8h_7yAJDbBBknOaskEtC4AQGWm1jId dnVSfxYD-WghRcdIOHKcGeMdV8xDoemlOvISfxYD-WglpgiNnEYnql.&priceTId=214784b d17475009279626648e7cbb&skuId=5825538911737&spm=a21n57.sem.item.4.150a3903i6ltg8& utparam=%7B%22aplus_abtest%22%3A%2275f152e772a17963e14d607916a2b322%22%7D&x xc=ad_ztc

华为 mate70Pro:

https://detail.tmall.com/item.htm?id=854501417778&ns=1&pisk=gG-icf0tcF7snBcdJHssrysVjn0 pBGsfZIEAMiCq865QWi36HtYHFIfthVs9-sAet1rtCddh3IvXWA_x1-26xQqYXfg63ojA0bh-20p _BisqwYbb9AN1nOPZDszaLG6YB45el0p6CswggYLE2-238Rw4usRVYwW5liWN0Z5EKtBa7i 5NuWzFU6SV0ZSVLPw5QPPN01k3LTBz7s7NgMPFUTNV0iR2LvXfTtSV0xNLI_A6wHlQ2 GJfZw4ieOQGad53qpthQlfKV1qqFHWMKsvalor2xObMYe0knL79uL_WXho0AGpk8MWJCcq h4dWk66dr4kjeppRAPeGb2wOHmZ-N5-kFThAHfGRtuuAljIbybw24DLvFsMXe4xU59Hf9ghb EeubAY3QPbe3I6ZIGE1-6s-q2appJfwtmZW5XWTsFedigxgJN3gJg8zPjao6EHHzbINWCK_3SA 9HQIWe06vD3PD7NdOinKv4blNWCK_HnKzzF796sx&priceTId=214784bd1747500927962664 8e7cbb&skuId=5823867018571&spm=a21n57.sem.item.1.150a3903i6ltg8&utparam=%7B%22ap lus_abtest%22%3A%22cc17f7da443fdfed09273e3712b02313%22%7D&xxc=ad_ztc

附录 2

介绍：基于 python3.10.11 的天猫网页评论数据爬取代码

```
from DrissionPage import ChromiumPage
import requests
import re
import json
import csv
import hashlib
import time
import random

def GetSign(data_time, pageNo):    #需根据实际网页和用户信息填写
```

```

ey = '2af7f8b47defed5b4dcbc039df633fc0'
eS = '12574478'
em='{\"showTrueCount\":false,\"auctionNumId\": \"854186606579\", \"pageNo
\":%s, \"pageSize\":20, \"rateType\": \"\", \"searchImpr\": \"-8\", \"orderTy
pe\": \"\", \"expression\": \"\", \"rateSrc\": \"pc_rate_list\"}' % pageNo
string = ey + "&" + str(data_time) + "&" + eS + "&" + em #加密变量
MD5 = hashlib.md5()
MD5.update(string.encode('utf-8'))
sign = MD5.hexdigest()
print(sign)
return sign, em

#创建文件对象
f = open('data.csv', mode='w', encoding='utf-8', newline='')
#字典写入
csv_writer = csv.DictWriter(f, fieldnames=['昵称', '产品', '评论'])
#写入表头
csv_writer.writeheader()
headers = {
    #Cokkie 用户信息，根据自身实际数值填写

'Cookie': 'lid=tb8610543266;havana_lgc_exp=1747346335550;wk_cookie2=126c
ddf7314cda7667665e3fdbbe3e647;wk_unb=UUpGRK%2FXwPlTyyqHaQ%3D%3D;dnk=tb86
10543266;uc1=existShop=false&cookie16=U%2BGCWk%2F74Mx5tgzv3dWpnjhPaQ%3D
%3D&cookie21=VT5L2FSpczFp&pas=0&cookie15=WqG3DMC9VAQiUQ%3D%3D&cookie14=
UoYajLZNHoEj3A%3D%3D;uc3=nk2=F5RNY%2BrfL001bsUC&id2=UUpGRK%2FXwPlTyyqHa
Q%3D%3D&lg2=URm48syIIVrSKA%3D%3D&vt3=F8dD2EXR0Igvbap2sXU%3D;tracknick=t
b8610543266;_l_g_=Ug%3D%3D;uc4=nk4=0%40FY4GtKHjW5JC3R0VcFj9YAdXL8fp8RA%
3D&id4=0%40U2gqy16ZhGJDwnVDpw24BHU1d8H70tq7;unb=2212249006984;lgc=tb861
0543266;cookie1=UUjQy46z0oVBGugN1vnUvg96UWzLPqTYApYJ0PaLTPw%3D;login=tr
ue;cookie17=UUpGRK%2FXwPlTyyqHaQ%3D%3D;cookie2=2de20f3d7b7b5e36482f7cad
3a022d84;_nk_=tb8610543266;sgcookie=E100AbqeSvLTI7%2BA303LcjDpvaddtrqtT
IC1p22B5Pus371fxpx4%2Bq8EGDo3LPCxMlJlRZeGbIpB7eBqFZMMQ4GVJZ9raZm19%2Fj%
2FQHE3%2FcSF0bfzo2Dg1hsqML29IFyGnEOD;cancelledSubSites=empty;sg=648;t=4
adfab0c5dd0c664cdc7e28a6c5051ac;csg=8a4895d1;sn=;_tb_token_=39336ebe651
0b;havana_sdkSilent=1747403532163;mtop_partitioned_detect=1;_m_h5_tk=2a
f7f8b47defed5b4dcbc039df633fc0_1747334886572;_m_h5_tk_enc=fbf1751607f47
19bcd35f6562378d20f;bxuab=0;x5sectag=117671;x5sec=7b2274223a31373437333
3303337362c22733b32223a2233326563353237613335643135623034222c2261736572
7665723b33223a22307c434d4c4b6d4d4547454f72736f6148352f2f2f2f774561447
a49794d5449794e446b774d4459354f4451374d79494a5932467763485636656d786c4d
4c375270717747227d;isg=BBoauUNQJjNldaqfRDCLnozsa8A8S541tdaRICST0a1417jR
DNunNR2kZmMLIxa9',

```

```

    'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/136.0.0.0 Safari/537.36'
}

for page in range(1,200):
    print(f'正在采集第{page}页的数据内容')
    #延时等待
    time.sleep(random.randint(1,2))
    #获取时间戳
    date_time = int(time.time() * 1000)
    #获取 sign 加密参数
    sign, em= GetSign(date_time, page)

url='https://h5api.m.tmall.com/h5/mtop.taobao.rate.detaillist.get/6.0/'

# 查询参数
data = {
    'jsv': '2.7.5',
    'appKey': '12574478',
    't': date_time,
    'sign': sign,
    'api': 'mtop.taobao.rate.detaillist.get',
    'v': '6.0',
    'isSec': '0',
    'ecode': '1',
    'timeout': '20000',
    'type': 'jsonp',
    'dataType': 'jsonp',
    'jsonpIncPrefix': 'pcdetail',
    'callback': 'mtopjsonpdetail19',
    'data': em,
}

# 发送请求
response = requests.get(url=url, params=data, headers=headers)
# 获取数据
# 获取响应的文本数据
text = response.text
print(text)
# 解析数据
# 提取 json 数据
info = re.findall('mtopjsonpdetail19\\((.*)', text)[0][:-1]
# 变成字典
json_data = json.loads(info)

```

```

# 根据键值对取值
rateList = json_data['data']['rateList']
for index in rateList:
    # 提取具体内容
    dit = {
        '昵称': index['userNick'],
        '产品': index['skuMap'],
        '评论': index['feedback']
    }
    # 写入数据
    csv_writer.writerow(dit)
    print(dit)

```

附录 3

介绍：基于 python3.10.11 的知乎页面评论数据爬取代码

```

from DrissionPage import ChromiumPage
from DataRecorder import Recorder

r =Recorder('zhihu.xlsx')
r.add_data(['content'])
r.record()
#实例化浏览器对象
dp = ChromiumPage()
# #访问网站，需要访问的网站
# dp.get('https://www.zhihu.com/question/669033840/answer/42067018827')
#监听数据包，实际 api 链接
dp.listen.start('https://www.zhihu.com/api/v4/comment_v5/answers/42067018827/root_comment?order_by')

for page in range(8): #抓取 8 页
    dp.scroll.to_bottom()
    resp = dp.listen.wait()
    json_data = resp.response.body
    data_list = json_data['data']
    for data in data_list:
        content = data['content']
        print(content)
        r.add_data([content])
        r.record()

```

附录 4

介绍：基于 python3.10.11 的评论内容情感分析及可视化

```

import pandas as pd
from snownlp import SnowNLP
import matplotlib.pyplot as plt
import seaborn as sns
import re
import os

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

INPUT_CSV = "data.csv"
OUTPUT_CSV = "sentiment_results.csv"
KEYWORDS = {
    "屏幕": ["屏幕", "显示", "色彩", "分辨率", "刷新率"],
    "续航": ["续航", "电池", "耗电", "充电", "电量"],
    "摄像头": ["摄像头", "拍照", "镜头", "像素", "对焦"],
    "性能": ["性能", "流畅", "卡顿", "发热", "处理器"],
    "外观": ["好看", "美丽", "漂亮", "炫酷", "外观"]
}

#数据预处理
def load_data():
    """加载并预处理数据"""
    if not os.path.exists(INPUT_CSV):
        raise FileNotFoundError(f"输入文件 {INPUT_CSV} 不存在！")

    try:
        df = pd.read_csv(INPUT_CSV)
        return df["评论"].tolist()
    except Exception as e:
        print(f"文件读取失败: {e}")
        exit()

def split_sentences(text):
    """中文分句处理"""
    sentences = re.split(r'[，。！？；、]', str(text))
    return [s.strip() for s in sentences if len(s.strip()) > 0]

def sentiment_analysis(comments):
    print("正在进行情感分析...")
    sentences = []
    original_comments = []
    for comment in comments:
        s_list = split_sentences(comment)

```

```

        sentences.extend(s_list)
        original_comments.extend([comment] * len(s_list))

# 计算情感得分
sentiment_scores = []
for s in sentences:
    try:
        score = SnowNLP(s).sentiments
        sentiment_scores.append(score)
    except:
        sentiment_scores.append(0.5)
return pd.DataFrame({
    "原始评论": original_comments,
    "分句": sentences,
    "情感得分": sentiment_scores
})

def dimension_analysis(df):
    """维度分类与分析"""
    # 维度分类
    def _assign_dimension(sentence):
        for dim, words in KEYWORDS.items():
            if any(word in sentence for word in words):
                return dim
        return "其他"

    df["设计维度"] = df["分句"].apply(_assign_dimension)
    filtered_df = df[df["设计维度"] != "其他"]

    # 综合指标计算
    dimension_stats = filtered_df.groupby("设计维度").agg(
        讨论量=("设计维度", "count"),
        平均情感=("情感得分", "mean"),
        情感中位数=("情感得分", "median")
    ).reset_index()

    dimension_stats["权重"] = (dimension_stats["讨论量"] /
    dimension_stats["讨论量"].sum()).round(3)
    dimension_stats = dimension_stats.sort_values("权重", ascending=False)

    return filtered_df, dimension_stats

def plot_sentiment_distribution(df):
    """情感得分分布直方图"""

```

```

plt.figure(figsize=(10, 6))
plt.hist(df["情感得分"], bins=20, range=(0, 1),
         edgecolor="black", alpha=0.7)
plt.title("情感得分分布 (0=负面, 1=正面)")
plt.xlabel("情感得分")
plt.ylabel("句子数量")
plt.savefig("sentiment_distribution.png", dpi=300)

def plot_dimension_boxplot(df):
    """设计维度箱线图"""
    plt.figure(figsize=(12, 6))
    sns.boxplot(x="设计维度", y="情感得分", data=df, palette="Set2")
    plt.xticks(rotation=45)
    plt.title("各维度情感得分分布")
    plt.tight_layout()
    plt.savefig("dimension_boxplot.png", dpi=300)

def plot_dimension_weights(stats_df):
    """用户偏好权重分布图"""
    plt.figure(figsize=(12, 6))
    bar = sns.barplot(x="设计维度", y="权重",
                     data=stats_df, palette="viridis")

    # 添加数据标签
    for p in bar.patches:
        height = p.get_height()
        bar.annotate(f"{height:.1%}",
                    (p.get_x() + p.get_width() / 2., height),
                    ha="center", va="center",
                    xytext=(0, 9),
                    textcoords="offset points")

    plt.ylim(0, stats_df["权重"].max() + 0.1)
    plt.title("用户偏好权重分布")
    plt.xticks(rotation=45)
    plt.savefig("dimension_weights.png", dpi=300)

def plot_correlation(stats_df):
    """权重-情感关联分析图"""
    plt.figure(figsize=(10, 6))
    scatter = sns.scatterplot(
        x="权重", y="平均情感",
        hue="设计维度", size="讨论量",
        sizes=(100, 500), palette="Dark2",

```



```

        data=stats_df
    )

    # 添加标注
    for line in stats_df.itertuples():
        plt.text(line.权重 + 0.01, line.平均情感 + 0.01,
                 line.设计维度, fontsize=9, alpha=0.8)

    plt.axhline(0.5, color="grey", linestyle="--")
    plt.axvline(stats_df["权重"].mean(), color="grey", linestyle="--")
    plt.title("用户偏好权重与情感得分关联分析")
    plt.xlabel("用户偏好权重")
    plt.ylabel("平均情感得分")
    plt.grid(alpha=0.3)
    plt.savefig("dimension_correlation.png", dpi=300)

# 主程序
if __name__ == "__main__":
    # 数据加载
    comments = load_data()

    # 情感分析
    result_df = sentiment_analysis(comments)
    result_df.to_csv(OUTPUT_CSV, index=False, encoding="utf_8_sig")
    print(f"结果已保存至 {OUTPUT_CSV}")

    # 维度分析
    filtered_df, stats_df = dimension_analysis(result_df)

    stats_df.to_csv("dimension_stats.csv", index=False, encoding="utf_8_sig")

    # 可视化
    plot_sentiment_distribution(result_df)
    plot_dimension_boxplot(filtered_df)
    plot_dimension_weights(stats_df)
    plot_correlation(stats_df)

    # 控制台输出
    print("\n 关键指标统计:")
    print(stats_df[["设计维度", "权重", "平均情感"]].to_string(index=False))
    print("\n 可视化图表已生成: ")
    print("- sentiment_distribution.png : 情感分布直方图")

```

```
print("- dimension_boxplot.png      : 维度箱线图")
print("- dimension_weights.png      : 用户偏好权重图")
print("- dimension_correlation.png   : 关联分析图")
```

附录 5

介绍：基于 python3.10.11 的词云图绘制

```
import jieba
import pandas
import pandas as pd
import wordcloud
from nltk.corpus import stopwords

df = pd.read_csv('data.csv')
content = ' '.join([i for i in df['评论']])
#分词处理
string = ' '.join(jieba.lcut(content))
#配置词云图
wc = wordcloud.WordCloud(
    width=1000,
    height=700,
    background_color='white', # 设置背景颜色
    font_path='msyh.ttc',
    stopwords={'了', '用', '还', '也', '都', '的', '手机', '我', '是', '还
是', '填写', '评价', '用户', '啊', '很', '好', '和', '没', '就是', '没有', '感觉
', 'mate70', '华为', '可以', '非常', '喜欢', '不错', '买', '就', '一直', '真的
', 'Mate', '在', '到', '特别', '支持', '能'}) #去除干扰词
wc.generate(string)
wc.to_file('cy.png')
# print(string)
```

附录 6

介绍：基于 python3.10.11 的问题二的求解

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize
import pandas as pd
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# 需求模型
# 参数设定
alpha = 50000      # 基础销量
beta1 = -5         # 自身价格系数
beta2 = 3          # 竞品价格系数
```

```

gamma1 = 100          # 配置评分系数
nu = 10               # 需求指数系数
mu = -5               # 供给指数系数

# 生成模拟数据
np.random.seed(42)
n_samples = 100
P_self = np.random.uniform(3000, 6000, n_samples)
P_comp = np.random.uniform(3000, 6000, n_samples)
config_score = np.random.normal(8, 2, n_samples)
D_demand = np.random.uniform(0.8, 1.2, n_samples)
S_supply = np.random.uniform(0.5, 1.5, n_samples)

# 计算销量（确保非负）
Q = alpha + beta1 * P_self * 0.6 + beta2 * P_comp + gamma1 * config_score
+ nu * D_demand + mu * S_supply
Q = np.clip(Q + np.random.normal(0, 1500, n_samples), 1000, None)

# 价格与销量关系可视化
plt.figure(figsize=(10, 6))
plt.scatter(P_self, Q, c='blue', alpha=0.6, label='实际数据')
price_range = np.linspace(3000, 8000, 100)
Q_pred = alpha + beta1 * price_range * 0.6 + beta2 * 5000 + gamma1 * 8 +
nu * 1 + mu * 1
plt.plot(price_range, Q_pred, 'r-', linewidth=2, label='理论需求曲线')
plt.title("价格与销量关系（实际数据 vs 理论模型）")
plt.xlabel("自身价格（元）")
plt.ylabel("销量")
plt.legend()
plt.grid(True)
plt.show()

# 价格弹性分析
# 计算价格弹性（点弹性公式）
def price_elasticity(P, Q, beta):
    return (beta * P) / Q

# 计算某价格点的弹性
P_sample = 5000
Q_sample = alpha + beta1 * P_sample * 0.6 + beta2 * 5000 + gamma1 * 8 + nu
* 1 + mu * 1
elasticity = price_elasticity(P_sample, Q_sample, beta1 * 0.6)
print(f"在价格{P_sample}元时，价格弹性为: {elasticity:.2f}")

```

```

# 产品线定价模型
theta1 = 2.0      # 基础版配置效用
theta2 = 3.0      # Pro 版配置效用
lambda_high = 0.001
lambda_low = 0.0008
C_base = 2500
C_pro = 4000
total_demand = int(np.mean(Q))

def profit(prices):
    P_base, P_pro = prices
    V_base = theta1 * 7 - lambda_high * P_base
    V_pro = theta2 * 9 - lambda_low * P_pro
    exp_v = np.exp([V_base, V_pro])
    P_choose_base = exp_v[0] / np.sum(exp_v)
    P_choose_pro = exp_v[1] / np.sum(exp_v)
    Q_base = total_demand * P_choose_base
    Q_pro = total_demand * P_choose_pro
    profit_total = (P_base - C_base) * Q_base + (P_pro - C_pro) * Q_pro
    return -profit_total

# 价格约束
initial_prices = [4000, 7000]
bounds = [(3000, 6000), (5000, 10000)]
result = minimize(profit, initial_prices, bounds=bounds)
optimal_prices = result.x

# 不同定价方案对比
def simulate_pricing_strategy(base_price_change=0, pro_price_change=0):
    P_base_new = optimal_prices[0] * (1 + base_price_change)
    P_pro_new = optimal_prices[1] * (1 + pro_price_change)
    V_base = theta1 * 7 - lambda_high * P_base_new
    V_pro = theta2 * 9 - lambda_low * P_pro_new
    exp_v = np.exp([V_base, V_pro])
    P_choose_base = exp_v[0] / np.sum(exp_v)
    P_choose_pro = exp_v[1] / np.sum(exp_v)
    Q_base = total_demand * P_choose_base
    Q_pro = total_demand * P_choose_pro
    profit_total = (P_base_new - C_base) * Q_base + (P_pro_new - C_pro) *
Q_pro
    return Q_base + Q_pro, profit_total

scenarios = {
    "原价策略": (0, 0),

```

```

    "基础版降价 5%": (-0.05, 0),
    "Pro 版降价 5%": (0, -0.05),
    "同时降价 5%": (-0.05, -0.05)
}

results = {}
for name, (change_base, change_pro) in scenarios.items():
    total_sales, total_profit = simulate_pricing_strategy(change_base,
change_pro)
    results[name] = {"销量": total_sales, "利润": total_profit}

# 可视化
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))
ax1.bar(results.keys(), [v[" 销 量 "] for v in results.values()],
color=['blue', 'orange', 'green', 'red'])
ax1.set_title("不同定价策略下的销量对比")
ax1.set_ylabel("销量")
ax1.tick_params(axis='x', rotation=45)
ax2.bar(results.keys(), [v[" 利 润 "] for v in results.values()],
color=['blue', 'orange', 'green', 'red'])
ax2.set_title("不同定价策略下的利润对比")
ax2.set_ylabel("利润（元）")
ax2.tick_params(axis='x', rotation=45)
plt.tight_layout()
plt.show()

# 不同配置版本成本与利润率对比
config_data = {
    "版本": ["基础版", "Pro 版"],
    "成本": [C_base, C_pro],
    "建议售价": [optimal_prices[0], optimal_prices[1]],
    "销量占比": [0.6, 0.4], # 假设基础版销量占比 60%, Pro 版 40%
    "利润率": [
        (optimal_prices[0] - C_base) / optimal_prices[0],
        (optimal_prices[1] - C_pro) / optimal_prices[1]
    ]
}

df_config = pd.DataFrame(config_data)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 6))
# 成本与售价对比
ax1.bar(df_config["版本"], df_config["成本"], width=0.4, label='成本',
align='edge')

```

```

ax1.bar(df_config["版本"], df_config["建议售价"], width=-0.4, label='建议
售价', align='edge')
ax1.set_title("不同版本成本与售价对比")
ax1.set_ylabel("价格（元）")
ax1.legend()
# 利润率与销量占比对比
ax2.bar(df_config["版本"], df_config["利润率"], width=0.4, label='利润率',
align='edge')
ax2.bar(df_config["版本"], df_config["销量占比"], width=-0.4, label='销量
占比', align='edge')
ax2.set_title("利润率与销量占比对比")
ax2.set_ylabel("比例")
ax2.legend()
plt.tight_layout()
plt.show()

# 输出结果
print("最优定价策略: ")
print(f"基础版价格: {optimal_prices[0]:.2f} 元, Pro 版价格:
{optimal_prices[1]:.2f}元\n")

print("不同定价方案效果: ")
for name, data in results.items():
    print(f"{name}: 销量={data['销量']:.0f}, 利润={data['利润']:.2f}元")

print("\n 不同配置版本经济指标: ")
print(df_config.round(2))

```

附录 7

介绍: 基于 python3.10.11 的问题二的可视化

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

def generate_historical_data(start_year=2023, months=24):
    # 初始化参数
    np.random.seed(42)
    date_range = pd.date_range(start=f"{start_year}-01-01",
periods=months, freq='M')

```

```

# 模拟基础趋势
base_price = np.linspace(6000, 4500, months)
comp_price = np.linspace(5800, 4200, months) + np.random.normal(0, 300,
months)
config_score = np.clip(np.sin(np.linspace(0, 3 * np.pi, months)) * 2 +
8, 6, 10)

# 生成动态数据
data = {
    "日期": date_range,
    "月度": np.arange(1, months + 1),
    "自身价格": base_price + np.random.normal(0, 200, months),
    "竞品价格": comp_price,
    "配置评分": config_score,
    "需求指数": np.clip(np.random.normal(1.0, 0.1, months), 0.8, 1.2),
    "供给指数": np.clip(np.random.normal(1.0, 0.2, months), 0.5, 1.5)
}

# 计算销量
df = pd.DataFrame(data)
df["销量"] = (
    50000 # 基础销量
    - 8 * df["自身价格"] * 0.6
    + 5 * df["竞品价格"]
    + 80 * df["配置评分"]
    + 30 * df["需求指数"]
    - 10 * df["供给指数"]
    + np.random.normal(0, 1500, months)
)

# 添加滞后效应
df["销量"] = df["销量"] + 0.3 * df["销量"].shift(1).fillna(0)
df["销量"] = np.clip(df["销量"], 10000, None).astype(int)

# 计算市场份额
total_market = df["销量"].mean() * 1.2
df["市场份额"] = df["销量"] / total_market

return df[["日期", "月度", "自身价格", "竞品价格", "配置评分", "需求指数",
"供给指数", "销量", "市场份额"]]

# 生成 24 个月的历史数据

```

```

history_df = generate_historical_data()

# 价格-销量趋势图
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 8), sharex=True)

# 价格趋势
ax1.plot(history_df["日期"], history_df["自身价格"], 'b-o', label="自身价格")
ax1.plot(history_df["日期"], history_df["竞品价格"], 'r--s', label="竞品价格")
ax1.set_ylabel("价格 (元)")
ax1.set_title("价格趋势对比")
ax1.legend()
ax1.grid(True)

# 销量趋势
ax2.bar(history_df["日期"], history_df["销量"], width=20, alpha=0.6, label="销量")
ax2.set_ylabel("销量")
ax2.set_title("月度销量趋势")
ax2.grid(True)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

# 价格-销量散点矩阵
from pandas.plotting import scatter_matrix

features = ["自身价格", "竞品价格", "配置评分", "销量"]
scatter_matrix(history_df[features], figsize=(12, 8), diagonal='kde', alpha=0.5)
plt.suptitle("价格-销量关系矩阵", y=1.02)
plt.show()

print("\n 历史数据框架示例: ")
print(history_df.head(5).round(2))

print("\n 数据统计描述: ")
print(history_df.describe().round(2))

```

附录 8

介绍：基于 python3.10.11 的问题三的求解

```
import numpy as np
```



```

import pandas as pd
import matplotlib.pyplot as plt
from pymoo.core.problem import Problem
from pymoo.algorithms.moo.nsga2 import NSGA2
from pymoo.optimize import minimize

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

#参数定义
design_params = {
    '摄像头': {'base': 48, 'min': 48, 'max': 64, 'cost': 80},
    '性能': {'base': 8.5, 'min': 8.5, 'max': 10.0, 'cost': 500},
    '外观': {'base': 7.2, 'min': 6.4, 'max': 9.0, 'cost': 300},
    '屏幕': {'base': 6.7, 'min': 6.0, 'max': 7.0, 'cost': 1000},
    '续航': {'base': 4500, 'min': 4500, 'max': 6400, 'cost': 0.2}
}

user_weights = np.array([0.289, 0.213, 0.206, 0.163, 0.13])
base_cost = 4000      # 基础款成本
base_price = 5499     # 基础款售价

#优化问题定义
class PhoneOptimizationProblem(Problem):
    def __init__(self):
        super().__init__(
            n_var=6,          # 5 参数 + 1 价格
            n_obj=2,          # 目标: 利润最大化, 成本最小化
            n_constr=3,       # 约束条件数量
            xl=self.get_lower_bounds(),
            xu=self.get_upper_bounds()
        )

    def get_lower_bounds(self):
        return [v['min'] for v in design_params.values()] + [base_price*0.8]
# 允许价格下探

    def get_upper_bounds(self):
        return [v['max'] for v in design_params.values()] + [base_price*1.5]
# 最高溢价 50%

    def _evaluate(self, X, out, *args, **kwargs):
        profits, costs = [], []
        g1, g2, g3 = [], [], []

```

```

for x in X:
    # 解析设计参数
    params = x[:5]
    price = x[5]

    # 计算参数提升比例
    improve_ratios = [(p - design_params[k]['base']) /
design_params[k]['base']
                        for p, k in zip(params, design_params.keys())]

    # 配置吸引力指数（考虑边际效应递减）
    attraction = np.sum(np.sqrt(improve_ratios) * user_weights) *
150

    # 计算成本（加入规模效应）
    delta_params = [
        params[0] - design_params['摄像头']['base'],
        (params[1] - design_params['性能']['base']) * 10,
        (params[2] - design_params['外观']['base']) * 10,
        (params[3] - design_params['屏幕']['base']) * 10,
        params[4] - design_params['续航']['base']
    ]

    cost_increment = sum([
        delta_params[0] * design_params['摄像头']['cost'] * 0.95,
        delta_params[1] * design_params['性能']['cost'] * 0.9,
        delta_params[2] * design_params['外观']['cost'],
        delta_params[3] * design_params['屏幕']['cost'],
        delta_params[4] * design_params['续航']['cost']
    ])

    total_cost = (base_cost + cost_increment) * 0.98 # 规模效应系
数

    # 销量预测模型（加入价格弹性增强）
    price_ratio = (price - base_price) / base_price
    sales = 10000 * (1 + attraction/250) * (1 - 2.2 * abs(price_ratio))
# 弹性系数提高到 2.2
    sales = max(sales, 2500) # 最低销量保障

    # 利润计算
    profit = (price - total_cost) * sales

```

```

# 目标函数
profits.append(-profit)
costs.append(total_cost)

# 约束条件
g1.append(price-1.2 * total_cost) # 售价不超过成本 1.2 倍
g2.append(18-attraction)          # 最低吸引力指数提高到 18
g3.append(base_price*0.8-price)    # 最低售价限制

out["F"] = np.column_stack([profits, costs])
out["G"] = np.column_stack([g1, g2, g3])

#优化求解
problem = PhoneOptimizationProblem()
algorithm = NSGA2(pop_size=100)
res = minimize(problem, algorithm, ('n_gen', 200), seed=1, verbose=False)

#结果分析
pareto_front = res.F
solutions = res.X

# 选择满足所有约束的最优解
valid_mask = np.all(res.G <= 0, axis=1)
valid_front = pareto_front[valid_mask]
valid_solutions = solutions[valid_mask]

# 按利润排序选择最优解
if valid_front.shape[0] > 0:
    best_idx = np.argmin(valid_front[:, 0])
    best_solution = valid_solutions[best_idx]
    best_profit = -valid_front[best_idx, 0]
    best_cost = valid_front[best_idx, 1]
else:
    raise ValueError("未找到满足所有约束的可行解")

# 结果展示
print("最优设计方案: ")
param_names = list(design_params.keys())
for i, name in enumerate(param_names):
    base = design_params[name]['base']
    improve = best_solution[i] - base
    print(f"{name}:          {base}          →          {best_solution[i]:.1f}
(+{improve/base*100:.1f}%)")

```

```

print(f"\n 建议定价: {best_solution[5]:.0f} 元")
print(f"预计成本: {best_cost:.0f} 元")
print(f"售价/成本比: {best_solution[5]/best_cost:.2f}")
print(f"预期利润: {best_profit/1e8:.2f} 亿元")

#可视化分析
plt.figure(figsize=(10, 6))
plt.scatter(-pareto_front[valid_mask,0]/1e8,
            pareto_front[valid_mask,1]/1e4,
            c=valid_solutions[:,5], cmap='viridis', alpha=0.6)
plt.colorbar(label='定价 (元)')
plt.title('Pareto 最优前沿 - 利润 vs 成本')
plt.xlabel('利润 (亿元)')
plt.ylabel('成本 (万元)')
plt.grid(True)

# 成本-价格关系验证
plt.figure(figsize=(10,6))
plt.scatter(valid_solutions[:,5], [x/1e4 for x in valid_front[:,1]],
            c=-valid_front[:,0]/1e8, cmap='plasma')
plt.plot([0, 10000], [0, 10000/1.2/1e4], 'r--', label='1.2 倍成本线')
plt.colorbar(label='利润 (亿元)')
plt.title('成本-价格关系验证')
plt.xlabel('定价 (元)')
plt.ylabel('成本 (万元)')
plt.legend()
plt.grid(True)

# 参数改进雷达图
def plot_radar_chart(solution):
    params = solution[:5]
    base_values = [design_params[k]['base'] for k in design_params]
    labels = list(design_params.keys())
    # 计算改进比例
    ratios = [(p - base)/base*100 for p, base in zip(params, base_values)]
    # 雷达图数据闭环
    angles = np.linspace(0, 2*np.pi, len(labels), endpoint=False).tolist()
    ratios += ratios[:1]
    angles += angles[:1]
    fig, ax = plt.subplots(figsize=(8,8), subplot_kw={'polar': True})
    ax.fill(angles, ratios, color='red', alpha=0.25)
    ax.plot(angles, ratios, color='red', linewidth=2, marker='o')
    ax.set_theta_offset(np.pi/2)
    ax.set_theta_direction(-1)
    ax.set_rlabel_position(0)

```

```

plt.xticks(angles[:-1], labels, fontsize=12)
plt.ylim(-15, 30) # 显示-15%到+30%的改进范围
ax.plot(angles, [0]*len(angles), '--', color='blue', alpha=0.5)
plt.title('设计参数改进雷达图 (%)', pad=20)
plt.show()
plot_radar_chart(best_solution)

# 参数改进对比柱状图
plt.figure(figsize=(12, 6))
param_names = list(design_params.keys())

# 计算基于参数范围的归一化值
base_normalized = []
optimized_normalized = []
for i, name in enumerate(param_names):
    min_val = design_params[name]['min']
    max_val = design_params[name]['max']
    base = design_params[name]['base']
    optimized = best_solution[i]

    # 计算基准值和优化值的归一化位置
    base_norm = (base - min_val) / (max_val - min_val)
    optimized_norm = (optimized - min_val) / (max_val - min_val)

    base_normalized.append(base_norm)
    optimized_normalized.append(optimized_norm)

x = np.arange(len(param_names))
width = 0.35

# 绘制双柱对比图
rects1 = plt.bar(x - width/2, base_normalized, width, label='基准值位置',
                 color='#2ecc71', edgecolor='black', alpha=0.8)
rects2 = plt.bar(x + width/2, optimized_normalized, width, label='优化值位置',
                 color='#e74c3c', edgecolor='black', alpha=0.8)

# 添加参数范围标注线
for i in range(len(param_names)):
    plt.hlines(y=1.0, xmin=i-0.4, xmax=i+0.4, colors='grey',
              linestyle='dashed', alpha=0.5)
    plt.text(i, 1.05, "参数上限", ha='center', fontsize=9, color='grey')

# 设置纵轴格式

```

```

plt.gca().yaxis.set_major_formatter(plt.FuncFormatter(lambda y, _:
f"{y:.0%}"))
plt.ylabel('参数范围位置 (min→max)')
plt.title('设计参数定位对比（基于参数范围的归一化）', pad=20, fontsize=14)
plt.xticks(x, param_names)
plt.legend(loc='upper center', bbox_to_anchor=(0.5, 1.15), ncol=2)

# 添加数据标签
def autolabel(rects):
    for rect in rects:
        height = rect.get_height()
        plt.annotate(f'{height:.0%}',
                     xy=(rect.get_x() + rect.get_width()/2, height),
                     xytext=(0, 3), textcoords="offset points",
                     ha='center', va='bottom', fontsize=9)

autolabel(rects1)
autolabel(rects2)

plt.ylim(0, 1.2)
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()

# 三维参数空间投影
from mpl_toolkits.mplot3d import Axes3D

fig = plt.figure(figsize=(12,8))
ax = fig.add_subplot(111, projection='3d')

# 选择三个核心参数展示
x = valid_solutions[:,0] # 摄像头
y = valid_solutions[:,1] # 性能
z = valid_solutions[:,3] # 屏幕

sc = ax.scatter(x, y, z, c=valid_solutions[:,5], cmap='plasma', s=50)

ax.set_xlabel('摄像头 (MP)')
ax.set_ylabel('性能评分')
ax.set_zlabel('屏幕尺寸 (in)')
plt.colorbar(sc, label='定价 (元)')
plt.title('三维参数空间投影')
plt.show()

plt.show()

```